ESTRUCTURAS DINÁMICAS DE DATOS

EN LENGUAJE C

Realizado por Ismael Camarero
icamarero@gmail.com

Forma parte de la web:
http://programandoenc.webcindario.com

Junio de 2005.

Generalidades:

Introducción:

Hasta ahora, hemos empleado diferentes tipos de datos: int, char, float… y agrupaciones de estos como pueden ser los arrays, struct… Sin embargo, pese a su utilidad, presentan un problema: deben ser declarados en todo su tamaño. Si declaramos un array como:

```
float array[100[100];
```

estamos declarando un array de 100x100 elementos flotantes que van a ocupar

100x100xsizeof(float) bytes de memoria.

El tamaño se decide en la declaración de la variable, no en el momento en que lo necesitamos.
Pero ¿qué sucede si sólo queremos guardar un elemento? Que desaprovechamos el resto de la memoria reservada para 10000 elementos float. Es un derroche que, aunque con las memorias actuales se pueda permitir, no debemos hacer.
Además ¿qué sucedería si en un momento necesitamos cambiar la cantidad de datos a almacenar?

Una de las aplicaciones más interesantes y potentes de la memoria dinámica y los punteros son las estructuras dinámicas de datos. Las estructuras básicas disponibles en C y C++ tienen una importante limitación: no pueden cambiar de tamaño durante la ejecución. Los arreglos están compuestos por un determinado número de elementos, número que se decide en la fase de diseño, antes de que el programa ejecutable sea creado.

En muchas ocasiones se necesitan estructuras que puedan cambiar de tamaño durante la ejecución del programa. Por supuesto, podemos hacer 'arrays' dinámicos, pero una vez creados, tu tamaño también será fijo, y para hacer que crezcan o diminuyan de tamaño, deberemos reconstruirlas desde el principio (funciones calloc(), malloc() y realloc())

Vamos a ver una nueva forma de gestionar datos: **ESTRUCTURAS DINAMICAS DE DATOS**

Las estructuras dinámicas nos permiten crear estructuras de datos que se adapten a las necesidades reales a las que suelen enfrentarse nuestros programas. Pero no sólo eso, como veremos, también nos permitirán crear estructuras de datos muy flexibles, ya sea en cuanto al orden, la estructura interna o las relaciones entre los elementos que las componen.

Las estructuras de datos están compuestas de otras pequeñas estructuras a las que llamaremos nodos o elementos, que agrupan los datos con los que trabajará nuestro programa y además uno o más punteros autoreferenciales, es decir, punteros a objetos del mismo tipo tipo.

Dentro de los datos de este tipo de datos podemos hablar de:

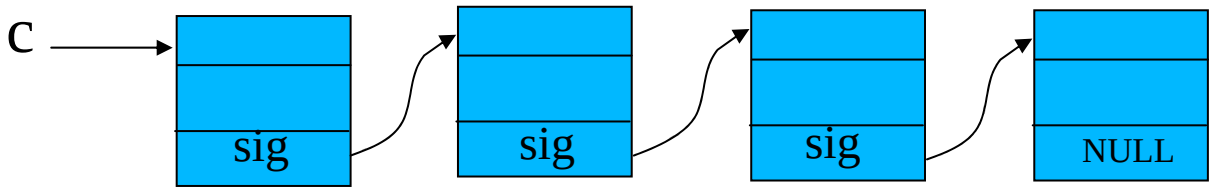
- Listas.
- Pilas.
- Colas.
- Árboles.
- Grafos

LISTAS ENLAZADAS

Una lista lineal enlazada es un conjunto de elementos u objetos de cualquier tipo, originariamente vacía que, durante la ejecución del programa va creciendo o decreciendo elemento a elemento según las necesidades previstas.

En una lista lineal cada elemento apunta al siguiente, es decir, cada electo tiene información de dónde esta el siguiente. Por este motivo también se le llama **lista enlazada.**

Gráficamente se puede representar en la forma:



Las estructuras dinámicas son una implementación de TDAs o TADs (Tipos Abstractos de Datos).

Dependiendo del **número de punteros** y de las **relaciones entre nodo**s, podemos distinguir varios tipos de estructuras dinámicas:

Listas simplemente enlazadas (o abiertas):

Cada elemento (nodo) sólo dispone de **un puntero**, que apuntará al siguiente elemento de la lista o valdrá **NULL** si es el último elemento. Sólo se pueden recorrer hacia delante.

PILAS:

Son un tipo especial de lista, conocidas como listas LIFO (Last In, First Out): el último en entrar es el primero en salir). Los elementos se "amontonan" o **apilan**, de modo que sólo el elemento que está encima de la pila puede ser leído, y sólo pueden añadirse elementos encima de la pila.

COLAS:

Otro tipo de listas, conocidas como listas FIFO (First In, First Out: El primero en entrar es el primero en salir). Los elementos se almacenan en fila, pero sólo pueden añadirse por un extremo y leerse por el otro.

LISTAS CIRCULARES:

También llamadas listas cerradas, son parecidas a las listas enlazadas, pero el último elemento apunta al primero. De hecho, en las listas circulares no puede hablarse de "primero" ni de "último". Cualquier nodo puede ser el nodo de entrada y salida. Se recorren siempre en el mismo sentido.

LISTAS DOBLEMENTE ENLAZADAS:

Cada elemento dispone de **dos punteros**, uno apunta al siguiente elemento y el otro al elemento anterior. Al contrario que las listas abiertas anteriores, estas listas pueden recorrerse en los dos sentidos.

ÁRBOLES:

Cada elemento dispone de **dos o más punteros**, pero las referencias nunca son a elementos anteriores, de modo que la estructura se ramifica y crece igual que un árbol.

ARBOLES BINARIOS:

Son árboles donde cada nodo sólo puede apuntar a dos nodos.

ARBOLES BINARIOS DE BÚSQUEDA (ABB)

Son árboles binarios ordenados. Desde cada nodo todos los nodos de una rama serán mayores, según la norma que se haya seguido para ordenar el árbol, y los de la otra rama serán menores.

ÁRBOLES AVL:

Son también árboles de búsqueda, pero su estructura está más optimizada para reducir los tiempos de búsqueda.

ARBOLES B:

Son estructuras más complejas, aunque también se trata de árboles de búsqueda, están mucho más optimizados que los anteriores.

TABLAS HASH:

Son estructuras auxiliares para ordenar listas.

GRAFOS:

Es el siguiente nivel de complejidad, podemos considerar estas estructuras como árboles no jerarquizados.

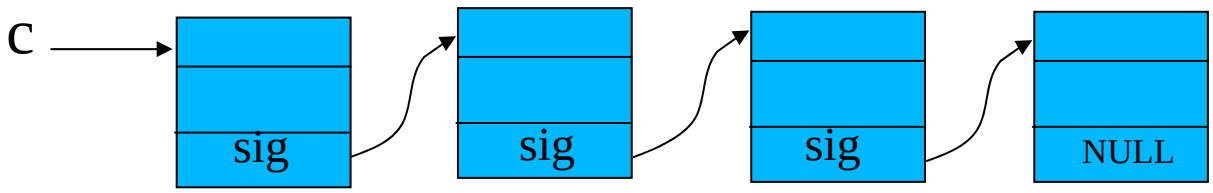
DICCIONARIOS:

Estructuras dinámicas en las que existen nodos de distintos tipos, en realidad no es obligatorio que las estructuras dinámicas estén compuestas por un único tipo de nodo, la flexibilidad y los tipos de estructuras sólo están limitados por tu imaginación como programador.

Listas simplemente enlazadas

La forma más simple de estructura dinámica es la lista **simplemente enlazada** o **lista abierta**.

En esta forma los nodos se organizan de modo que cada uno apunta al siguiente, y el último no apunta a nada, es decir, el puntero del nodo siguiente vale NULL:



La anterior es una lista simplemente enlazada que consta de 4 elementos (nodos).

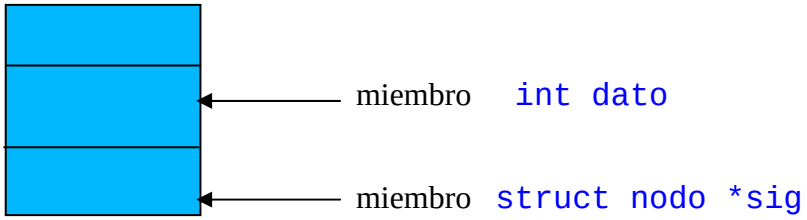
Para crear un alista debemos definir la clase de elementos que van a formar parte de la misma. Un tipo de dato genérico podría ser:

```
struct nodo
 {
   int dato;
   struct nodo *sig;
 };
```

El miembro de la estructura "sig", puede apuntar a un objeto del tipo nodo. De este modo, cada nodo puede usarse como un ladrillo para construir listas de datos, y cada uno mantendrá ciertas relaciones con otros nodos.

En el ejemplo, cada elemento de la lista sólo contiene un dato de tipo entero, pero en la práctica **no hay límite** en cuanto a la complejidad de los datos a almacenar. Se pueden diseñar datos struct que actúen como nodos tan complejos como se desee.

El nodo anterior se representará así (no consideramos el miembro datos, sólo el miembro sig, que ahora apunta a NULL



Para acceder a un nodo de la estructura sólo necesitaremos un puntero a un nodo. En el ejemplo anterior declaramos una variable de estructura, que va a ser un puntero a dicha estructura.

Mediante typedef declaramos un nuevo tipo de dato:

```
typedef struct s
 {
   int dato;
   struct s *siguiente;
 } elemento;

elemento *c; //puntero a nodo
```

Ahora `elemento` es un tipo de dato, sinónimo de `struct s`. El miembro `sig` ha sido declarado como puntero al dato `struct s` y no como puntero a tipo de dato `elemento`, ya que en ese instante, elemento no estaba aún definido.

Por tanto, **elemento** es el tipo para declarar nodos. Ahora es un tipo de dato. La variable *c es un puntero al tipo de dato elemento

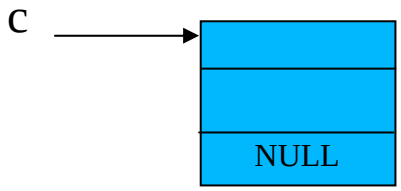
En las listas simples enlazadas existe un nodo especial: **el primero**. Normalmente diremos que nuestra lista es un **puntero a ese primer nodo** y llamaremos a ese nodo **cabeza de la lista**. Eso es porque mediante ese único puntero podemos acceder a toda la lista.

Es muy importante que nuestro programa **nunca pierda el valor del puntero al primer elemento**, ya que **si no existe ninguna copia de ese valor**, y se pierde, **será imposible acceder** al nodo y no podremos utilizar la información almacenada ni liberar el espacio de memoria que ocupa.

Cuando el puntero que usamos para acceder a la lista vale **NULL**, diremos que la **lista está vacía**:

```
*c = NULL;  //lista vacía
```

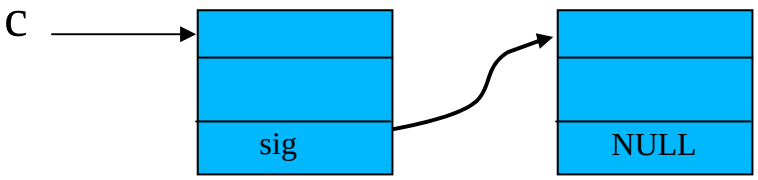
Inicialmente `*sig` apunta a NULL. Lo representamos gráficamente en la forma:



La variable puntero **c** apunta al primer nodo. Esta lista consta de un nodo.

Si ahora hacemos que el miembro *sig guarde la dirección de memoria del siguiente nodo (diremos que apunte al siguiente nodo), tenemos dos nodos localizables mediante

p y **p->siguiente**



Operaciones básicas con listas:

Con las listas se pueden realizar las siguientes operaciones básicas:

a) Crear lista.
b) Añadir o insertar elementos.
c) Buscar o localizar elementos.
d) Borrar elementos.
e) Moverse a través de una lista.
f) Ordenar una lista.

Cada una de estas operaciones tendrá varios casos especiales, por ejemplo, no será lo mismo insertar un nodo en una lista vacía, o al principio de una lista no vacía, o la final, o en una posición intermedia.

a) Crear Lista.

Si queremos crear una lista, podemos emplear la técnica de reservar memoria dinámicamente. Para ello escribimos una función llamada nuevo_elemento:

```
elemento *nuevo_ elemento(void)
  {
     elemento *q;
     q = (elemento *)malloc(sizeof(elemento));
     if(!q)
         perror("No se ha reservado memoria para el  nuevo nodo");
         return q; //devuelve la direcc. de memomria reservada
  }
```

La función principal tomará la forma:

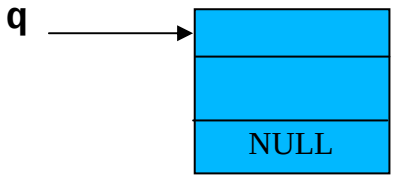
```
int main()
{
    elemento *q; //puntero a un nodo
    q = NULL; //lista vacía

    //Reserva dinámica de mem
    q = nuevo_elemento(); //q apunta al nodo recién creado
    q->sig = NULL; //fin de lista

    //operaciones. . .

   //Liberación de memoria reservada
   free(q);
 }
```

Gráficamente:



Cada vez que deseemos crear un nuevo nodo, debemos llamar a la función `nuevo_elemento()` para realizar la reserva de memoria.

b) Añadir o insertar elementos en una lista enlazada:

Veremos primero los casos sencillos y finalmente construiremos un algoritmo genérico para la inserción de elementos en una lista.

Insertar un elemento en una lista vacía:

Este es el caso más sencillo. Equivale a crear una lista, como en el caso a). Partiremos de que ya tenemos el nodo a insertar (creado en la llamada a la función `nuevo_elemento()` y, por supuesto un puntero que apunte a él, además el puntero a la lista valdrá NULL:

El proceso es muy simple, bastará con que:

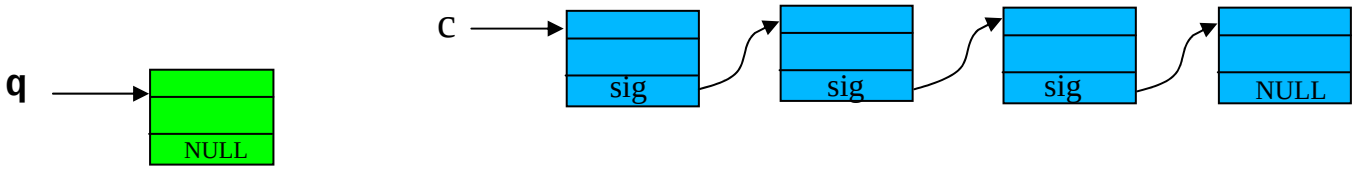
```
q = nuevo_elemento();
q->siguiente  = NULL;
```

Insertar un elemento en la primera posición de una lista:

Podemos considerar el caso anterior como un caso particular de éste, la única diferencia es que en el caso anterior la lista es una lista vacía, pero siempre podemos, y debemos considerar una lista vacía como una lista.

Por tanto, ahora **la lista ya existe.**

De nuevo partiremos de un nodo a insertar, con un puntero que apunte a él (**q**), y de una lista, en este caso no vacía, apuntada por **c**:



Hemos creado q mediante:

```
q = nuevo_elemento();
```

Apuntamos q->sig a c:

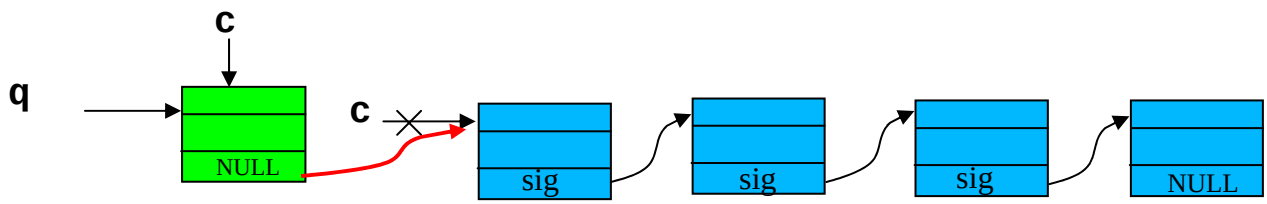
```
q->sig = c;
```

Ahora reapuntamos c a q:

```
c = q;
```

Es fundamental no alterar el orden de las operaciones.

Gráficamente, tenemos:



Este sistema de inserción permite generalizar un método:

```
/*Programa que crea una lista lineal y permite insertar datos numéricos enteros en el primer nodo. Para finalizar la introducción de datos, pulsar EOF (en windows CRT + Z; en Linux CTRL +D) Antes finalizar imprime los datos introducidos, en orden inverso al de introducción como consecuencia de que siempre insertamos el nuevo nodo en la cabecera de la lista.*/
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>

struct datos
{
	int num;
	struct datos *sig;
};

typedef datos *p;

//prototipos de funciones a utilizar.
datos *nuevo_nodo(void);
void error(void);
int main()
```

```
{
    int contador = 0, n = 0;
    datos *q = NULL, *c=NULL;  //Lista vacía
    printf("\nDato numero %d: ", contador++);
    while( scanf("%d", &n) != EOF)
     {
       q = nuevo_nodo();
       q->num = n; //num es un miembro de la estructura
       q->sig = c;
       c = q;
       printf("\nDato numero %d: ", contador++);
     } //fin de while
   do
     {
       printf("%d\t", c->num);
     }while((c = c->sig));
   free(q);
   getch();
}

//cuerpo de la función para reservar memoria
datos *nuevo_nodo(void)
{
      datos *q = (datos *)malloc(sizeof(datos));
      if(!q)
        error();
      return q;
}

void error(void)
{
     perror("Memoria no reservada");
     getch();
     exit(1);
}
```

El orden de los elementos en la lista, es el inverso del orden en que has sido introducidos.

Insertar un elemento en la última posición de una lista:

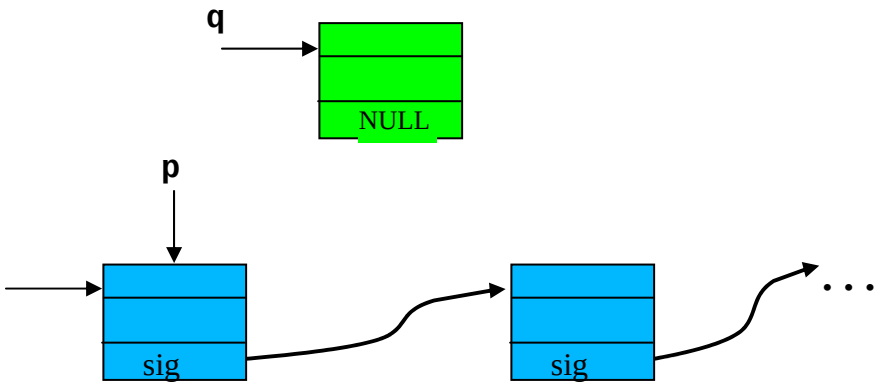
Este es otro caso especial. Para este caso partiremos de una lista no vacía:

El proceso en este caso tampoco es excesivamente complicado:

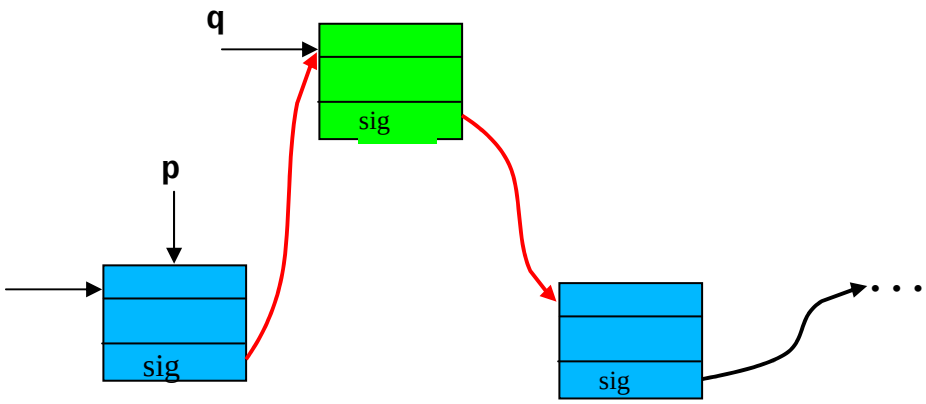
Necesitamos un puntero que señale al último elemento de la lista. La manera de conseguirlo es empezar por el primero y avanzar hasta que el nodo que tenga como siguiente el valor NULL (es decir, el último).
Hacer que nodo->siguiente sea NULL.
Hacer que ultimo->siguiente sea nodo.

Insertar un elemento a continuación de un nodo cualquiera de una lista:

De nuevo podemos considerar el caso anterior como un caso particular de este. Ahora el nodo "anterior" será aquel (**p**) a continuación del cual insertaremos el nuevo nodo:



El proceso gráfico es:



El fragmento de código en lenguaje C que explica el proceso es:

```
....
q = nuevo_nodo();
q->num = n; //n es el entero que insertamos
q->sig = p->sig;
p->sig = q;
```

c) Localizar elementos en una lista enlazada:

Muy a menudo necesitaremos recorrer una lista, ya sea buscando un valor particular o un nodo concreto. Las listas abiertas sólo pueden recorrerse en un sentido, ya que cada nodo apunta al siguiente, pero no se puede obtener, por ejemplo, un puntero al nodo anterior desde un nodo cualquiera si no se empieza desde el principio.

Para recorrer una lista procederemos siempre del mismo modo, usaremos un puntero auxiliar **q** como índice:

```
. . . .
q = c;
printf("¿Valor a localizar?");
scanf("%d", &x);
while(q != NULL && q->num != x)
  q = q->sig;
. . . .
```

El anterior razonamiento se puede emplear para visualizar todos los elementos de una lista:

```
. . . .
q = c;
while(q != NULL)
 {
   printf("%d\t", q->num);
   q = q->sig;
 }
. . . .
```

Supongamos que sólo queremos mostrar los valores hasta que encontremos uno que sea mayor que 100, podemos sustituir el bucle por:

```
.  .  .  .
q = c;
while(q != NULL && q->num <100)
 {
   printf("%d\t", q->num);
   q = q->sig;
 }
.  .  .  .
```

Si analizamos la condición del bucle, tal vez encontremos un posible error: ¿Qué pasaría si ningún valor es mayor que 100, y alcancemos el final de la lista?. Podría pensarse que cuando `q` sea NULL, si intentamos acceder a `q->num` se producirá un error.

En general eso será cierto, no puede accederse a punteros nulos. Pero en este caso, ese acceso está dentro de una condición y forma parte de una expresión "and". Recordemos que cuando se evalúa una expresión "and", se comienza por la izquierda, y la evaluación se abandona cuando una de las expresiones resulta falsa, de modo que la expresión "`q->num < 100`" nunca se evaluará si `q` es NULL.

Si hubiéramos escrito la condición al revés, el programa nunca funcionaría bien. Esto es algo muy importante cuando se trabaja con punteros.

d) Borrar elementos de una lista enlazada:

De nuevo podemos encontrarnos con varios casos, según la posición del nodo a eliminar.

d1) Eliminar el primer nodo de una lista abierta:

Es el caso más simple. Partiremos de una lista con uno o más nodos, y usaremos un puntero auxiliar, **q**:

Hacemos que **q** apunte al primer elemento de la lista, es decir a **c**.
Asignamos a **c** la dirección del segundo nodo de la lista: `c = c->siguiente.`
Liberamos la memoria asignada al primer nodo, q, el que queremos eliminar: `free(q);`

Si no guardamos el puntero al primer nodo antes de actualizar **c**, después nos resultaría imposible liberar la memoria que ocupa. Si liberamos la memoria antes de actualizar **c**, perderemos el puntero al segundo nodo.

Si la lista sólo tiene un nodo, el proceso es también válido, ya que el valor de c->siguiente es NULL, y después de eliminar el primer nodo la lista quedará vacía, y el valor de `c` será NULL.

De hecho, el proceso que se suele usar para borrar listas completas es eliminar el primer nodo hasta que la lista esté vacía.

d2) Eliminar un nodo cualquiera de una lista enlazada:

En todos los demás casos, eliminar un nodo se puede hacer siempre del mismo modo. Supongamos que tenemos una lista con al menos dos elementos, y un puntero **`p`** al nodo **anterior** al que queremos eliminar. Y un puntero auxiliar **`q.`**

Hacemos que **`q`** apunte al nodo que queremos borrar (el siguiente a **`p`**):

```
q = p->sig;
```

Ahora, asignamos como nodo siguiente del nodo anterior, el siguiente al que queremos eliminar:

```
p->sig  =  q->sig;
```

Eliminamos la memoria asociada al nodo que queremos eliminar.

```
free(q);
```

Si el nodo a eliminar es el último, es procedimiento es igualmente válido, ya que **`p`** pasará a ser el último, y `p->sig` valdrá NULL.

d3) Borrar todos los elementos de una lista

Borrar todos los elementos de una lista equivale a liberar la memoria reservada para cada uno de los elementos de la misma. Si el primer nodo está apuntado por **`c,`** empleamos el puntero auxiliar **`q:`**

```
q = c;  //salvamos el puntero a la lista
while(q != NULL)
  {
    c = c->sig;
    free(q);
    q = c;
  }
```

Hay que observar que antes de borrar el elemento apuntado por **`q`**, hacemos que c apunte al siguiente elemento ya que si no perdemos el resto de la lista

e) Moverse a través de una lista lineal simplemente enlazada:

Sólo hay un modo de moverse a través de una lista abierta, **hacia adelante.**

Aún así, a veces necesitaremos acceder a determinados elementos de una lista abierta. Veremos ahora como acceder a los más corrientes: el primero, el último, el siguiente y el anterior.

Primer elemento (nodo) de una lista:

El primer elemento es el más accesible, ya que es a ese a que apunta el puntero que define la lista. Para obtener un puntero al primer elemento bastará con copiar el puntero **c.**

Elemento siguiente a uno cualquiera:

Supongamos que tenemos un puntero **q** que señala a un elemento de una lista. Para obtener un puntero al siguiente bastará con asignarle el campo "siguiente" del nodo **q**:

```
q = q->siguiente;
```

Elemento anterior a uno cualquiera:

Ya hemos dicho que **no es posible retroceder en una lista** l**ineal simplemente enlazada**, de modo que para obtener un puntero al nodo anterior a uno dado tendremos que partir del primero, e ir avanzando hasta que el nodo siguiente sea precisamente nuestro nodo.

Último elemento de una lista:

Para obtener un puntero al último elemento de una lista partiremos de un nodo cualquiera, por ejemplo el primero, y avanzaremos hasta que su nodo siguiente sea NULL.

Saber si una lista está vacía:

Basta con comparar el puntero a la cabecera de la lista, **c**, con NULL, si **`c vale`** NULL la **lista** está **vacía.**

Borrar una lista completa:

El algoritmo genérico para borrar una lista completa consiste simplemente en borrar el primer elemento sucesivamente mientras la lista no esté vacía.

f) Ordenar una lista enlazada:

Supongamos que queremos construir una lista para almacenar números enteros, pero de modo que siempre esté ordenada de menor a mayor. Para hacer la prueba añadiremos los valores 20, 10, 40, 30. De este modo tendremos todos los casos posibles. Al comenzar, el primer elemento se introducirá en una lista vacía, el segundo se insertará en la primera posición, el tercero en la última, y el último en una posición intermedia.

Insertar un elemento en una lista vacía es equivalente a insertarlo en la primera posición. De modo que no incluiremos una función para asignar un elemento en una lista vacía, y haremos que la función para insertar en la primera posición nos sirva para ese caso también.

Algoritmo de inserción:

El primer paso es crear un nodo para el dato que vamos a insertar.

Si Lista es NULL, o el valor del primer elemento de la lista es mayor que el del nuevo, insertaremos el nuevo nodo en la primera posición de la lista.

En caso contrario, buscaremos el lugar adecuado para la inserción, tenemos un puntero "anterior". Lo inicializamos con el valor de Lista, y avanzaremos mientras anterior->siguiente no sea NULL y el dato que contiene anterior->siguiente sea menor o igual que el dato que queremos insertar.

Ahora ya tenemos anterior señalando al nodo adecuado, así que insertamos el nuevo nodo a continuación de él.

El código en lenguaje C de la función que emplearemos es:

```
void insertar(Lista *lista, int v)
{
   pNodo nuevo, anterior;

   /* Crear un nodo nuevo */
   nuevo = (pNodo)malloc(sizeof(tipoNodo));
   nuevo->valor = v;

   /* Si la lista está vacía */
   if(listaVacia(*lista) || (*lista)->valor > v) {
```

```
      /* Añadimos la lista a continuación del nuevo nodo */
      nuevo->siguiente = *lista;
      /* Ahora, el comienzo de nuestra lista es en nuevo nodo */
      *lista = nuevo;
   }
   else {
      /* Buscar el nodo de valor menor a v */
      anterior = *lista;
      /* Avanzamos hasta el último elemento o hasta que el siguiente tenga
         un valor mayor que v */
      while(anterior->siguiente && anterior->siguiente->valor <= v)
         anterior = anterior->siguiente;
      /* Insertamos el nuevo nodo después del nodo anterior */
      nuevo->siguiente = anterior->siguiente;
      anterior->siguiente = nuevo;
   }
}
```

Algoritmo para borrar un elemento:

Después probaremos la función para buscar y borrar, borraremos los elementos 10, 15, 45, 30 y 40, así probaremos los casos de borrar el primero, el último y un caso intermedio o dos nodos que **no** existan.

Recordemos que para eliminar un nodo necesitamos disponer de un puntero al nodo anterior.

Lo primero será localizar el nodo a eliminar, si es que existe. Pero sin perder el puntero al nodo anterior. Partiremos del nodo primero, y del valor NULL para anterior. Y avanzaremos mientras nodo no sea NULL o mientras que el valor almacenado en nodo sea menor que el que buscamos.
Ahora pueden darse tres casos:
Que el nodo sea NULL, esto indica que todos los valores almacenados en la lista son menores que el que buscamos y el nodo que buscamos no existe. Retornaremos sin borrar nada.
Que el valor almacenado en nodo sea mayor que el que buscamos, en ese caso también retornaremos sin borrar nada, ya que esto indica que el nodo que buscamos no existe.
Que el valor almacenado en el nodo sea igual al que buscamos.
De nuevo existen dos casos:
Que anterior sea NULL. Esto indicaría que el nodo que queremos borrar es el primero, así que modificamos el valor de Lista para que apunte al nodo siguiente al que queremos borrar.
Que anterior no sea NULL, el nodo no es el primero, así que asignamos a anterior->siguiente la dirección de nodo->siguiente.
Después, liberamos la memoria de nodo. La función que emplearemos es la siguiente

```
void borrar(Lista *lista, int v)
{
   pNodo anterior, nodo;

   nodo = *lista;
   anterior = NULL;
   while(nodo && nodo->valor < v) {
      anterior = nodo;
      nodo = nodo->siguiente;
   }
```

```
   if(!nodo || nodo->valor != v) return;
   else { /* Borrar el nodo */
      if(!anterior) /* Primer elemento */
         *lista = nodo->siguiente;
      else  /* un elemento cualquiera */
         anterior->siguiente = nodo->siguiente;
      free(nodo);
   }
}
```

Código completo del ejemplo:

```
#include <stdlib.h>
#include <stdio.h>

struct nodo {
   int valor;
   struct nodo *siguiente;
};//tipoNodo;

typedef  struct nodo tipoNodo;
typedef tipoNodo *pNodo;
typedef tipoNodo *Lista;

/* Funciones con listas: */
void insertar(Lista *l, int v);
void borrar(Lista *l, int v);
int listaVacia(Lista l);
void borrarLista(Lista *);
void mostrarLista(Lista l);

int main()
{
   Lista lista = NULL;
   pNodo p;

   insertar(&lista, 20);
   mostrarLista(lista);
   insertar(&lista, 10);
   mostrarLista(lista);
   insertar(&lista, 40);
   mostrarLista(lista);
```

```
   insertar(&lista, 30);
   mostrarLista(lista);

   puts("\nLista completa:");
   mostrarLista(lista);

   puts("\nBorramos elemento a elemento");
   borrar(&lista, 10);
   mostrarLista(lista);
   borrar(&lista, 15);  //  este elemento no existe
   mostrarLista(lista);
   borrar(&lista, 45);//no existe
   mostrarLista(lista);
   borrar(&lista, 30);
   mostrarLista(lista);
   borrar(&lista, 40);
   mostrarLista(lista);
   //borrar(&lista, 20);
   borrarLista(&lista);
   mostrarLista(lista);

   system("PAUSE");
   return 0;
}

void insertar(Lista *lista, int v)
{
   pNodo nuevo, anterior;

   /* Crear un nodo nuevo */
   nuevo = (pNodo)malloc(sizeof(tipoNodo));
   nuevo->valor = v;

   /* Si la lista está vacía */
   if(listaVacia(*lista) || (*lista)->valor > v)
     {
        /* Añadimos la lista a continuación del nuevo nodo */
        nuevo->siguiente = *lista;
        /* Ahora, el comienzo de nuestra lista es en nuevo nodo */
        *lista = nuevo;
```

```
      }
   else
      {
        /* Buscar el nodo de valor menor a v */
        anterior = *lista;
        /* Avanzamos hasta el último elemento o hasta que el siguiente tenga
         un valor mayor que v */

        while(anterior->siguiente && anterior->siguiente->valor <= v)
           anterior = anterior->siguiente;

        /* Insertamos el nuevo nodo después del nodo anterior */
        nuevo->siguiente = anterior->siguiente;
        anterior->siguiente = nuevo;
       }
}

void borrar(Lista *lista, int v)
{
   pNodo anterior, nodo;

   nodo = *lista;
   anterior = NULL;
   while(nodo && nodo->valor < v)
    {
      anterior = nodo;
      nodo = nodo->siguiente;
    }
   if(!nodo || nodo->valor != v)
     return;
   else
     { /* Borrar el nodo */
       if(!anterior) /* Primer elemento */
          *lista = nodo->siguiente;
       else  /* un elemento cualquiera */
          anterior->siguiente = nodo->siguiente;
       free(nodo);
    }
}
```

```
int listaVacia(Lista lista)
{
   return (lista == NULL);
}

void borrarLista(Lista *lista)
{
   pNodo nodo;

   while(*lista)
    {
      nodo = *lista;
      *lista = nodo->siguiente;
      free(nodo);
    }
}
void mostrarLista(Lista lista)
{
   pNodo nodo = lista;

   if(listaVacia(lista))
     printf("Lista vacia\n");
   else
     {
       while(nodo)
        {
          printf("%d -> ", nodo->valor);
          nodo = nodo->siguiente;
        }
      printf("\n");
     }
}
```

Pilas:

Definición de pila:

Una pila es un tipo especial de lista abierta en la que sólo se pueden **insertar** y **eliminar** nodos en uno de los extremos de la lista. Estas operaciones se conocen como "**push**" y "**pop**", respectivamente "empujar" y "tirar". Además, las escrituras de datos siempre son inserciones de nodos, y las lecturas siempre eliminan el nodo leído.

Estas características implican un comportamiento de lista **LIFO** (Last In First Out), el último en entrar es el primero en salir.

El símil del que deriva el nombre de la estructura es una pila de platos. Sólo es posible añadir platos en la parte superior de la pila, y sólo pueden tomarse del mismo extremo.

Los tipos que definiremos normalmente para manejar pilas serán casi los mismos que para manejar listas, tan sólo cambiaremos algunos nombres:

```
typedef struct _nodo {
   int dato;
   struct _nodo *siguiente;
} tipoNodo;

typedef tipoNodo *pNodo;
typedef tipoNodo *Pila;
```

tipoNodo es el tipo para declarar nodos, evidentemente.

pNodo es el tipo para declarar punteros a un nodo.

Pila es el tipo para declarar pilas.

Es evidente, que una pila es una lista abierta. Así que sigue siendo muy importante que nuestro programa **nunca pierda el valor del puntero al primer elemento**, igual que pasa con las listas abiertas.

Teniendo en cuenta que las inserciones y borrados en una pila se hacen siempre en un extremo, lo que consideramos como el primer elemento de la lista es en realidad el último elemento de la pila.

Operaciones básicas con pilas:

Las pilas tienen un conjunto de operaciones muy limitado, sólo permiten las operaciones de "push" y "pop":

Push: Añadir un elemento al final de la pila.
Pop: Leer y eliminar un elemento del final de la pila.

Las operaciones con pilas son muy simples, no hay casos especiales, salvo que la pila esté vacía.

Push en una pila vacía:

Partiremos de que ya tenemos el nodo a insertar y, por supuesto un puntero que apunte a él, además el puntero a la pila valdrá NULL:

El proceso es muy simple, bastará con que:

nodo->siguiente apunte a NULL.
Pila apunte a nodo.

Push en una pila no vacía:

Podemos considerar el caso anterior como un caso particular de éste, la única diferencia es que podemos y debemos trabajar con una pila vacía como con una pila normal.

De nuevo partiremos de un **nodo a insertar**, con un puntero que apunte a él, y de una pila, en este caso no vacía:

El proceso sigue siendo muy sencillo:

Hacemos que nodo->siguiente apunte a Pila.
Hacemos que Pila apunte a nodo.

Pop, leer y eliminar un elemento:

Ahora sólo existe un caso posible, ya que sólo podemos leer desde un extremo de la pila.

Partiremos de una pila con uno o más nodos, y usaremos un puntero auxiliar, nodo:

Hacemos que nodo apunte al primer elemento de la pila, es decir a Pila.
Asignamos a Pila la dirección del segundo nodo de la pila: Pila->siguiente.
Guardamos el contenido del nodo para devolverlo como retorno de la función (recordemos que la operación pop equivale a leer y borrar.)
Liberamos la memoria asignada al primer nodo, el que queremos eliminar.

Si la pila sólo tiene un nodo, el proceso sigue siendo válido, ya que el valor de Pila->siguiente es NULL, y después de eliminar el último nodo la pila quedará vacía, y el valor de Pila será NULL.

Ejemplo de pila en C:

Supongamos que queremos construir una pila para almacenar números enteros. Haremos pruebas intercalando varios "push" y "pop", y comprobando el resultado.

Algoritmo de la función "push":

Creamos un nodo para el valor que colocaremos en la pila.
Hacemos que nodo->siguiente apunte a Pila.
Hacemos que Pila apunte a nodo.

```
void Push(Pila *pila, int v)
{
   pNodo nuevo;

   /* Crear un nodo nuevo */
   nuevo = (pNodo)malloc(sizeof(tipoNodo));
   nuevo->valor = v;

   /* Añadimos la pila a continuación del nuevo nodo */
   nuevo->siguiente = *pila;
   /* Ahora, el comienzo de nuestra pila es en nuevo nodo */
   *pila = nuevo;
}
```

Algoritmo de la función "pop":

Hacemos que nodo apunte al primer elemento de la pila, es decir a Pila.
Asignamos a Pila la dirección del segundo nodo de la pila: Pila->siguiente.

Guardamos el contenido del nodo para devolverlo como retorno, recuerda que la operación pop equivale a leer y borrar.

Liberamos la memoria asignada al primer nodo, el que queremos eliminar.

```
int Pop(Pila *pila)
{
   pNodo nodo; /* variable auxiliar para manipular nodo */
   int v;      /* variable auxiliar para retorno */

   /* Nodo apunta al primer elemento de la pila */
   nodo = *pila;
   if(!nodo)
      return 0; /* Si no hay nodos en la pila retornamos 0 */

   /* Asignamos a pila toda la pila menos el primer elemento */
   *pila = nodo->siguiente;

   /* Guardamos el valor de retorno */
   v = nodo->valor;

   /* Borrar el nodo */
```

```
   free(nodo);
   return v;
}
```

Código completo del ejemplo:

```
#include <stdlib.h>
#include <stdio.h>

typedef struct _nodo {
   int valor;
   struct _nodo *siguiente;
} tipoNodo;

typedef tipoNodo *pNodo;
typedef tipoNodo *Pila;

/* Funciones con pilas: */
void Push(Pila *l, int v);
int Pop(Pila *l);

int main()
{
   Pila pila = NULL;

   Push(&pila, 20);
   Push(&pila, 10);
   printf("%d, ", Pop(&pila));
   Push(&pila, 40);
   Push(&pila, 30);

   printf("%d, ", Pop(&pila));
   printf("%d, ", Pop(&pila));
   Push(&pila, 90);
   printf("%d, ", Pop(&pila));
   printf("%d\n", Pop(&pila));

   system("PAUSE");
   return 0;
}

void Push(Pila *pila, int v)
{
   pNodo nuevo;

   /* Crear un nodo nuevo */
   nuevo = (pNodo)malloc(sizeof(tipoNodo));
   nuevo->valor = v;

   /* Añadimos la pila a continuación del nuevo nodo */
   nuevo->siguiente = *pila;
   /* Ahora, el comienzo de nuestra pila es en nuevo nodo */
   *pila = nuevo;
}

int Pop(Pila *pila)
```

```
{
   pNodo nodo; /* variable auxiliar para manipular nodo */
   int v;      /* variable auxiliar para retorno */

   /* Nodo apunta al primer elemento de la pila */
   nodo = *pila;
   if(!nodo) return 0; /* Si no hay nodos en la pila retornamos 0 */
   /* Asignamos a pila toda la pila menos el primer elemento */
   *pila = nodo->siguiente;
   /* Guardamos el valor de retorno */
   v = nodo->valor;
   /* Borrar el nodo */
   free(nodo);
   return v;
}
```

Ejemplo 2 de Pilas:

Se trata de un programa que simula una calculadora que realiza las opeaciones +, -, * y /, empleando la **notación polaca inversa**: pimero se introducen los operandos (números) y después el operador que indica la operación a realizar:

```
#include <stdio.h>
#include <stdlib.h>


typedef struct datos nodo;
struct datos /* estructura de un elemento de la pila */
  {
     double dato;
     nodo *siguiente;
  };

/* Prototipos de funciones */
void push(nodo **p, double x); // añadir un dato a la pila
double pop(nodo **p); // sacar un dato de la pila
void error(void);
nodo *nuevo_elemento(void);  //reserva dinámica de memoria

int  main()
 {
   nodo *q,*cima = NULL;
   double a, b;
   char op[81];

   printf("Calculadora con las operaciones: + - ^ /\n");
   printf("Los datos serán introducidos de la forma:\n");
   printf(">operando 1\n");
   printf(">operando 2\n");
   printf("operador\n\n");
   printf("Para salir pulse q\n\n");
   do
    {
      printf(">  ");
      gets(op); /* leer un operando o un operador */
```

```
      switch (*op)
       {
        case '+' :
          b = pop(&cima);
          a = pop(&cima);
          printf("%g\n", a + b);
          push(&cima, a+b);
        break;

        case '-' :
          b = pop(&cima);
          a = pop(&cima);
          printf("%g\n", a - b);
          push( &cima, a-b);
        break;

        case '*':
          b = pop(&cima);
          a = pop(&cima);
          printf("%g\n", a * b);
          push( &cima, a*b);
         break;

        case '/':
          b = pop(&cima);
          a = pop(&cima);
          if(b==0)
            {
              printf("Division por CERO");
              break;
            }
          printf("%g\n", a / b);
          push( &cima, a/b);
        break;

        default:
          push(&cima, atof(op));
       }
    }while(*op != 'q');

//liberamos memoria

  q = cima;

  while(q != NULL)
   {
     cima = cima->siguiente;
     free (q);
     q = cima;
   }
 } //fin de main()

 /*Añadir un dato a la pila*/
void push(nodo **p, double x)
{
  nodo *q, *cima;
  cima = *p;
  q = nuevo_elemento();
```

```
  q->dato = x;
  q->siguiente = cima;
  cima = q;
  *p=cima;
}

//recuperar de la cima

double pop(nodo **p)
 {
   nodo *cima;
   double x;
   cima= *p;
   if(cima ==NULL)
    {
     printf("ERRRRORRRRR");
      return 0;
    }
   else
    {
      x=cima->dato;
      *p=cima->siguiente;
      free(cima);
      return x;
    }
}

void error(void)
 {
    perror("Mem no reservada");
    exit(0);
 }

nodo *nuevo_elemento(void)
 {
    nodo *q = (nodo *)malloc(sizeof(nodo));
    if(!q)
      error();
    return (q);
 }
```

Colas:

Una cola es un **tipo especial de lista enalazada** en la que sólo se pueden **insertar** nodos en uno de los extremos de la lista y sólo se pueden **elimin**ar nodos en el otro. Además, como sucede con las pilas, las escrituras de datos siempre son inserciones de nodos, y las lecturas siempre eliminan el nodo leído.

Este tipo de lista es conocido como lista **FIFO** (First In First Out), el primero en entrar es el primero en salir.

El símil cotidiano es una cola para comprar, por ejemplo, las entradas del cine. Los nuevos compradores sólo pueden colocarse al final de la cola, y sólo el primero de la cola puede comprar la entrada.

El nodo típico para construir pilas es el mismo que vimos en los capítulos anteriores para la construcción de listas y pilas:

Los tipos que definiremos normalmente para manejar colas serán casi los mismos que para manejar listas y pilas, tan sólo cambiaremos algunos nombres:

```
typedef struct _nodo
{
   int dato;
   struct _nodo *siguiente;
} tipoNodo;

typedef tipoNodo *pNodo;
typedef tipoNodo *Cola;
```

tipoNodo es el tipo para declarar nodos, evidentemente.

pNodo es el tipo para declarar punteros a un nodo.

Cola es el tipo para declarar colas.

Es evidente, que una cola es una lista abierta. Así que sigue siendo muy importante que nuestro programa **nunca pierda el valor del puntero al primer element**o, igual que pasa con las listas abiertas. Además, debido al funcionamiento de las colas, también deberemos mantener un puntero para el último elemento de la cola, que será el punto donde insertemos nuevos nodos.

Teniendo en cuenta que las lecturas y escrituras en una cola se hacen siempre en extremos distintos, lo más fácil será insertar nodos por el final, a continuación del nodo que no tiene nodo siguiente, y leerlos desde el principio, hay que recordar que leer un nodo implica eliminarlo de la cola.

Operaciones básicas con colas:

De nuevo nos encontramos ante una estructura con muy pocas operaciones disponibles. Las colas sólo permiten **añadir** y **leer** elementos:

Añadir: Inserta un elemento al final de la cola.
Leer: Lee y elimina un elemento del principio de la cola.

Añadir un elemento:

Las operaciones con colas son muy sencillas, prácticamente no hay casos especiales, salvo que la cola esté vacía.

Añadir elemento en una cola vacía:

Partiremos de que ya tenemos el **nodo a insertar** y, por supuesto un **puntero** que apunte a él, además los **punteros** que definen la cola, primero y ultimo que valdrán NULL:

El proceso es muy simple, bastará con que:

nodo->siguiente apunte a NULL.
Y que los punteros primero y ultimo apunten a nodo.

Añadir elemento en una cola no vacía:

De nuevo partiremos de un nodo a insertar, con un puntero que apunte a él, y de una cola, en este caso, al no estar vacía, los punteros primero y ultimo no serán nulos:

El proceso sigue siendo muy sencillo:

Hacemos que nodo->siguiente apunte a NULL.
Después que ultimo->siguiente apunte a nodo.
Y actualizamos ultimo, haciendo que apunte a nodo.

Añadir elemento en una cola, caso general:

Para generalizar el caso anterior, sólo necesitamos añadir una operación:

Hacemos que nodo->siguiente apunte a NULL.
Si ultimo no es NULL, hacemos que ultimo->siguiente apunte a nodo.
Y actualizamos ultimo, haciendo que apunte a nodo.
Si primero es NULL, significa que la **cola estaba vacía**, así que haremos que primero apunte también a nodo.

Leer un elemento en una cola:

Recordemos que **leer** un elemento de una cola, implica eliminarlo.

Ahora también existen dos casos, que la cola tenga un solo elemento o que tenga más de uno.

Usaremos un puntero a un nodo auxiliar:

Hacemos que `nodo` apunte al primer elemento de la cola, es decir a `primero`.
Asignamos a primero la dirección del segundo nodo de la pila: `primero->siguiente`.
Guardamos el contenido del `nodo` para devolverlo como retorno, recuerda que la operación de lectura en colas implica también borrar.
Liberamos la memoria asignada al primer nodo, el que queremos eliminar.

Leer un elemento en una cola con un solo elemento:

También necesitamos un puntero a un nodo auxiliar:

Hacemos que `nodo` apunte al primer elemento de la pila, es decir a `primero`.
Asignamos NULL a `primero`, que es la dirección del `segundo` nodo teórico de la cola: `primero->siguiente`.

Guardamos el contenido del nodo para devolverlo como retorno, recuerda que la operación de lectura en colas implica también borrar.
Liberamos la memoria asignada al primer nodo, el que queremos eliminar.
Hacemos que `ultimo` apunte a `NULL`, ya que la lectura ha dejado la cola vacía.

Leer un elemento en una cola, caso general:

Hacemos que nodo apunte al primer elemento de la pila, es decir a `primero`.
Asignamos a primero la dirección del segundo nodo de la pila: `primero->siguiente`.
Guardamos el contenido del nodo para devolverlo como retorno, recuerda que la operación de lectura en colas implica también borrar.
Liberamos la memoria asignada al primer nodo, el que queremos eliminar.
Si primero es NULL, hacemos que ultimo también apunte a NULL, ya que la lectura ha dejado la cola vacía.

Ejemplo de cola en C:

Construiremos una cola para almacenar números enteros. Haremos pruebas insertando varios valores y leyéndolos alternativamente para comprobar el resultado.

Algoritmo de la función "Anadir":

Creamos un nodo para el valor que colocaremos en la cola.
Hacemos que `nodo->siguiente` apunte a `NULL`.
Si "ultimo" no es NULL, hacemos que `ultimo->siguiente` apunte a `nodo`.
Actualizamos "ultimo" haciendo que apunte a nodo.
Si "primero" es NULL, hacemos que apunte a nodo.

```
void Anadir(pNodo *primero, pNodo *ultimo, int v)
{
   pNodo nuevo;
```

```
   /* Crear un nodo nuevo */
   nuevo = (pNodo)malloc(sizeof(tipoNodo));
   nuevo->valor = v;

  /* Este será el último nodo, no debe tener siguiente */
   nuevo->siguiente = NULL;

/* Si la cola no estaba vacía, añadimos el nuevo a continuación de ultimo */
   if(*ultimo)
      (*ultimo)->siguiente = nuevo;

   /* Ahora, el último elemento de la cola es el nuevo nodo */
   *ultimo = nuevo;

   /*Si primero es NULL,la cola estaba vacía, ahora primero apuntará también al nuevo nodo */
   if(!*primero)
      *primero = nuevo;
}
```

Algoritmo de la función "leer":

Hacemos que `nodo` apunte al primer elemento de la cola, es decir a `primer`o.
Asignamos a primero la dirección del segundo nodo de la cola: `primero->siguiente`.
Guardamos el contenido del nodo para devolverlo como retorno, recuerda que la operación de lectura equivale a leer y borrar.
Liberamos la memoria asignada al primer nodo, el que queremos eliminar.
Si primero es NULL, haremos que último también apunte a NULL, ya que la cola habrá quedado vacía.

```
int Leer(pNodo *primero, pNodo *ultimo)
  {
   pNodo nodo; /* variable auxiliar para manipular nodo */
   int v;      /* variable auxiliar para retorno */

   /* Nodo apunta al primer elemento de la pila */
   nodo = *primero;
   if(!nodo)
     return 0; /* Si no hay nodos en la pila retornamos 0 */

   /* Asignamos a primero la dirección del segundo nodo */
   *primero = nodo->siguiente;

   /* Guardamos el valor de retorno */
   v = nodo->valor;

   /* Borrar el nodo */
   free(nodo);

   /* Si la cola quedó vacía, ultimo debe ser NULL también*/
   if(!*primero) *ultimo = NULL;
     return v;
}
```

Código completo del ejemplo:

Tan sólo nos queda escribir una pequeña prueba para verificar el funcionamiento de las colas:

```
#include <stdlib.h>
#include <stdio.h>

typedef struct _nodo {
   int valor;
   struct _nodo *siguiente;
} tipoNodo;

typedef tipoNodo *pNodo;

/* Funciones con colas: */
void Anadir(pNodo *primero, pNodo *ultimo, int v);
int Leer(pNodo *primero, pNodo *ultimo);

int main()
{
   pNodo primero = NULL, ultimo = NULL;

   Anadir(&primero, &ultimo, 20);
   printf("Añadir(20)\n");
   Anadir(&primero, &ultimo, 10);
   printf("Añadir(10)\n");
   printf("Leer: %d\n", Leer(&primero, &ultimo));
   Anadir(&primero, &ultimo, 40);
   printf("Añadir(40)\n");
   Anadir(&primero, &ultimo, 30);
   printf("Añadir(30)\n");
   printf("Leer: %d\n", Leer(&primero, &ultimo));
   printf("Leer: %d\n", Leer(&primero, &ultimo));
   Anadir(&primero, &ultimo, 90);
   printf("Añadir(90)\n");
   printf("Leer: %d\n", Leer(&primero, &ultimo));
   printf("Leer: %d\n", Leer(&primero, &ultimo));

   system("PAUSE");
   return 0;
}

void Anadir(pNodo *primero, pNodo *ultimo, int v)
{
   pNodo nuevo;

   /* Crear un nodo nuevo */
   nuevo = (pNodo)malloc(sizeof(tipoNodo));
   nuevo->valor = v;
   /* Este será el último nodo, no debe tener siguiente */
   nuevo->siguiente = NULL;
   /* Si la cola no estaba vacía, añadimos el nuevo a continuación de ultimo */
   if(*ultimo) (*ultimo)->siguiente = nuevo;
   /* Ahora, el último elemento de la cola es el nuevo nodo */
   *ultimo = nuevo;
```

```
   /* Si primero es NULL, la cola estaba vacía, ahora primero apuntará también al
nuevo nodo */
   if(!*primero) *primero = nuevo;
}

int Leer(pNodo *primero, pNodo *ultimo)
{
   pNodo nodo; /* variable auxiliar para manipular nodo */
   int v;      /* variable auxiliar para retorno */

   /* Nodo apunta al primer elemento de la pila */
   nodo = *primero;
   if(!nodo) return 0; /* Si no hay nodos en la pila retornamos 0 */
   /* Asignamos a primero la dirección del segundo nodo */
   *primero = nodo->siguiente;
   /* Guardamos el valor de retorno */
   v = nodo->valor;
   /* Borrar el nodo */
   free(nodo);
   /* Si la cola quedó vacía, ultimo debe ser NULL también*/
   if(!*primero) *ultimo = NULL;
   return v;
}
```

Listas circulares:

Una lista circular es una **lista lineal** en la que el **último nodo a punta al primero.**

Las listas circulares evitan excepciones en las operaciones que se realicen sobre ellas. No existen casos especiales, cada nodo siempre tiene uno anterior y uno siguiente.

En algunas listas circulares se añade un nodo especial de cabecera, de ese modo se evita la única excepción posible, la de que la lista esté vacía.

Los tipos que definiremos normalmente para manejar listas cerradas son los mismos que para para manejar listas abiertas:

```
typedef struct _nodo
{
   int dato;
   struct _nodo *siguiente;
} tipoNodo;

typedef tipoNodo *pNodo;
typedef tipoNodo *Lista;
```

tipoNodo es el tipo para declarar nodos, evidentemente.

pNodo es el tipo para declarar punteros a un nodo.

Lista es el tipo para declarar listas, tanto abiertas como circulares. En el caso de las circulares, apuntará a un nodo cualquiera de la lista.

A pesar de que las listas circulares simplifiquen las operaciones sobre ellas, también introducen algunas complicaciones. Por ejemplo, en un proceso de búsqueda, no es tan sencillo dar por terminada la búsqueda cuando el elemento buscado no existe.

Por ese motivo se suele resaltar un nodo en particular, que no tiene por qué ser siempre el mismo. Cualquier nodo puede cumplir ese propósito, y puede variar durante la ejecución del programa.

Otra alternativa que se usa a menudo, y que simplifica en cierto modo el uso de listas circulares es crear un nodo especial de hará la función de nodo cabecera. De este modo, la lista nunca estará vacía, y se eliminan casi todos los casos especiales.

Operaciones básicas con listas circulares:

A todos los efectos, las listas circulares son como las listas abiertas en cuanto a las operaciones que se pueden realizar sobre ellas:

a) Añadir o insertar elementos.
b) Buscar o localizar elementos.
c) Borrar elementos.
d) Moverse a través de la lista, siguiente.

Cada una de estas operaciones podrá tener varios casos especiales, por ejemplo, tendremos que tener en cuenta cuando se inserte un nodo en una lista vacía, o cuando se elimina el único nodo de una lista.

a) Añadir un elemento

El único caso especial a la hora de insertar nodos en listas circulares es cuando la lista esté vacía.

a1) Añadir elemento en una lista circular vacía:

Partiremos de que ya tenemos el nodo a insertar y, por supuesto un puntero que apunte a él, además el puntero que define la lista, que valdrá NULL:

El proceso es muy simple, bastará con que:

`lista` apunta a `nodo`.
`lista->siguiente` apunte a nodo.

a2) Añadir elemento en una lista circular no vacía:

De nuevo partiremos de un nodo a insertar, con un puntero que apunte a él, y de una lista, en este caso, el puntero no será nulo:

El proceso sigue siendo muy sencillo:

Hacemos que `nodo->siguiente` apunte a `lista->siguiente`.
Después que `lista->siguiente` apunte a nodo.

a3) Añadir elemento en una lista circular, caso general:

Para generalizar los dos casos anteriores, sólo necesitamos añadir una operación:

Si lista está vacía hacemos que lista apunte a nodo.
Si lista no está vacía, hacemos que `nodo->siguiente` apunte a `lista->siguiente.`
Después que `lista->siguiente` apunte a `nodo`.

b) Buscar un elemento en una lista circular:

A la hora de buscar elementos en una lista circular sólo hay que tener una precaución, **es necesario almacenar el puntero del nodo en que se empezó la búsqueda**, para poder detectar el caso en que no exista el valor que se busca. Por lo demás, la búsqueda es igual que en el caso de las listas abiertas, salvo que podemos empezar en cualquier punto de la lista.

c) Borrar un elemento de una lista circular:

Para ésta operación podemos encontrar tres casos diferentes:

Eliminar un nodo cualquiera, que no sea el apuntado por lista.
Eliminar el nodo apuntado por lista, y que no sea el único nodo.
Eliminar el único nodo de la lista.

En el primer caso necesitamos localizar el **nodo anterio**r al que queremos borrar. Como el principio de la lista puede ser cualquier nodo, haremos que sea precisamente lista quien apunte al nodo anterior al que queremos eliminar.

Esto elimina la excepción del segundo caso, ya que lista nunca será el nodo a eliminar, salvo que sea el único nodo de la lista.

Una vez localizado el nodo anterior y apuntado por lista, hacemos que `lista->siguiente` apunte a `nodo->siguiente`. Y a continuación borramos nodo.

En el caso de que sólo exista un nodo, será imposible localizar el nodo anterior, así que simplemente eliminaremos el nodo, y haremos que `lista` valga `NULL`.

c1) Eliminar un nodo en una lista circular con más de un elemento:

Consideraremos los dos primeros casos como uno sólo.

El primer paso es conseguir que `lista` apunte al `nodo` anterior al que queremos eliminar. Esto se consigue haciendo que lista valga `lista->siguiente` mientras `lista->siguiente` sea distinto de `nodo`.
Hacemos que `lista->siguiente` apunte a `nodo->siguiente`.
Eliminamos el `nodo`.

c2) Eliminar el único nodo en una lista circular:

Este caso es mucho más sencillo. Si `lista` es el único nodo de una lista circular:

Borramos el `nodo` apuntado por `lista`.

Hacemos que `lista = NULL`

Otro algoritmo para borrar nodos:

Existe un modo alternativo de eliminar un nodo en una lista circular con más de un nodo.

Supongamos que queremos eliminar un nodo apuntado por nodo:

Copiamos el contenido del `nodo->siguiente` sobre el contenido de `nodo`.
Hacemos que `nodo->siguiente` apunte a `nodo->siguiente->siguiente`.
Eliminamos `nodo->siguiente.`
Si lista es el `nodo->siguiente`, hacemos `lista = nodo`.

Este método también funciona con listas circulares de un sólo elemento, salvo que el nodo a borrar es el único nodo que existe, y hay que hacer que `lista = NULL.`

Ejemplo de lista circular en C:

Construiremos una lista cerrada para almacenar números enteros. Haremos pruebas insertando varios valores, buscándolos y eliminándolos alternativamente para comprobar el resultado.

Algoritmo de la función "Insertar":

Si lista está vacía hacemos que `lista` apunte a `nodo`.
Si lista no está vacía, hacemos que `nodo->siguiente` apunte a `lista->siguiente`.
Después que `lista->siguiente` apunte a `nodo.`

```
void Insertar(Lista *lista, int v)
{
   pNodo nodo;

   // Creamos un nodo para el nuvo valor a insertar
   nodo = (pNodo)malloc(sizeof(tipoNodo));
   nodo->valor = v;

   // Si la lista está vacía, la lista será el nuevo nodo
   // Si no lo está, insertamos el nuevo nodo a continuación del apuntado
   // por lista
   if(*lista == NULL)
     *lista = nodo;
   else nodo->siguiente = (*lista)->siguiente;
   // En cualquier caso, cerramos la lista circular
   (*lista)->siguiente = nodo;
}
```

Algoritmo de la función "Borrar":

¿Tiene la lista un único nodo?
SI:
Borrar el nodo lista.
Hacer `lista = NULL.`
NO:
Hacemos `lista->siguiente = nodo->siguiente.`
Borramos nodo.

```
void Borrar(Lista *lista, int v)
{
   pNodo nodo;

   nodo = *lista;

   // Hacer que lista apunte al nodo anterior al de valor v
   do
     {
      if((*lista)->siguiente->valor != v)
          *lista = (*lista)->siguiente;
     } while((*lista)->siguiente->valor != v && *lista != nodo);
      // Si existe un nodo con el valor v:
   if((*lista)->siguiente->valor == v)
     {
      // Y si la lista sólo tiene un nodo
      if(*lista == (*lista)->siguiente)
```

```
        {
         // Borrar toda la lista
         free(*lista);
         *lista = NULL;
       }
      else
         {
         // Si la lista tiene más de un nodo, borrar el nodo  de valor v
         nodo = (*lista)->siguiente;
         (*lista)->siguiente = nodo->siguiente;
         free(nodo);
      }
   }
}
```

Código completo del ejemplo:

```
#include <stdlib.h>
#include <stdio.h>

typedef struct _nodo {
   int valor;
   struct _nodo *siguiente;
} tipoNodo;

typedef tipoNodo *pNodo;
typedef tipoNodo *Lista;

// Funciones con listas:
void Insertar(Lista *l, int v);
void Borrar(Lista *l, int v);
void BorrarLista(Lista *);
void MostrarLista(Lista l);

int main()
{
   Lista lista = NULL;
   pNodo p;

   Insertar(&lista, 10);
   Insertar(&lista, 40);
   Insertar(&lista, 30);
   Insertar(&lista, 20);
   Insertar(&lista, 50);

   MostrarLista(lista);

   Borrar(&lista, 30);
   Borrar(&lista, 50);

   MostrarLista(lista);

   BorrarLista(&lista);
   system("PAUSE");
   return 0;
```

```
}

void Insertar(Lista *lista, int v)
{
   pNodo nodo;

   // Creamos un nodo para el nuvo valor a insertar
   nodo = (pNodo)malloc(sizeof(tipoNodo));
   nodo->valor = v;

   // Si la lista está vacía, la lista será el nuevo nodo
   // Si no lo está, insertamos el nuevo nodo a continuación del apuntado
   // por lista
   if(*lista == NULL) *lista = nodo;
   else nodo->siguiente = (*lista)->siguiente;
   // En cualquier caso, cerramos la lista circular
   (*lista)->siguiente = nodo;
}

void Borrar(Lista *lista, int v)
{
   pNodo nodo;

   nodo = *lista;

   // Hacer que lista apunte al nodo anterior al de valor v
   do {
      if((*lista)->siguiente->valor != v) *lista = (*lista)->siguiente;
   } while((*lista)->siguiente->valor != v && *lista != nodo);
   // Si existe un nodo con el valor v:
   if((*lista)->siguiente->valor == v) {
      // Y si la lista sólo tiene un nodo
      if(*lista == (*lista)->siguiente) {
         // Borrar toda la lista
         free(*lista);
         *lista = NULL;
      }
      else {
         // Si la lista tiene más de un nodo, borrar el nodo  de valor v
         nodo = (*lista)->siguiente;
         (*lista)->siguiente = nodo->siguiente;
         free(nodo);
      }
   }
}

void BorrarLista(Lista *lista)
{
   pNodo nodo;

   // Mientras la lista tenga más de un nodo
   while((*lista)->siguiente != *lista) {
      // Borrar el nodo siguiente al apuntado por lista
      nodo = (*lista)->siguiente;
      (*lista)->siguiente = nodo->siguiente;
      free(nodo);
   }
   // Y borrar el último nodo
```

```
   free(*lista);
   *lista = NULL;
}

void MostrarLista(Lista lista)
{
   pNodo nodo = lista;

   do {
      printf("%d -> ", nodo->valor);
      nodo = nodo->siguiente;
   } while(nodo != lista);
   printf("\n");
}

}
```

Listas doblemente enlazadas:

Una lista **doblemente enlazada es una lista lineal en la que cada nodo tiene dos enlaces**, uno al nodo siguiente, y otro al anterior.

Las listas doblemente enlazadas no necesitan un nodo especial para acceder a ellas, pueden recorrerse en ambos sentidos a partir de cualquier nodo, esto es porque a partir de cualquier nodo, siempre es posible alcanzar cualquier nodo de la lista, hasta que se llega a uno de los extremos.

El nodo típico es el mismo que para construir las listas que hemos visto, salvo que tienen otro puntero al nodo anterior:

Para C, y basándonos en la declaración de nodo que hemos visto más arriba, trabajaremos con los siguientes tipos:

```
typedef struct _nodo {
   int dato;
   struct _nodo *siguiente;
   struct _nodo *anterior;
} tipoNodo;

typedef tipoNodo *pNodo;
typedef tipoNodo *Lista;
```

tipoNodo es el tipo para declarar nodos, evidentemente.

pNodo es el tipo para declarar punteros a un nodo.

Lista es el tipo para declarar listas abiertas doblemente enlazadas. También es posible, y potencialmente útil, crear listas doblemente enlazadas y circulares.

El movimiento a través de listas doblemente enlazadas es más sencillo, y como veremos las operaciones de búsqueda, inserción y borrado, también tienen más ventajas.

Operaciones básicas con listas doblemente enlazadas:

De nuevo tenemos el mismo repertorio de operaciones sobre este tipo listas:

a) Añadir o insertar elementos.
b) Buscar o localizar elementos.
c) Borrar elementos.
d) Moverse a través de la lista, siguiente y anterior.

a) Añadir un elemento:

Vamos a intentar ver todos los casos posibles de inserción de elementos en listas doblemente enlazadas.

a1) Añadir elemento en una lista doblemente enlazada vacía:

Partiremos de que ya tenemos el nodo a insertar y, por supuesto un puntero que apunte a él, además el puntero que define la lista, que valdrá NULL:

El proceso es muy simple, bastará con que:

`lista` apunta a `nodo`.
`lista->siguiente` y `lista->anterior` apunten a `NULL`.

a2) Insertar un elemento en la primera posición de la lista:

Partimos de una lista no vacía. Para simplificar, consideraremos que lista apunta al primer elemento de la lista doblemente enlazada:

El proceso es el siguiente:

`nodo->siguiente` debe apuntar a `Lista`.
`nodo->anterior` apuntará a `Lista->anterio`r.
`Lista->anterior` debe apuntar a `nodo`.

Recuerda que Lista no tiene por qué apuntar a ningún miembro concreto de una lista doblemente enlazada, cualquier miembro es igualmente válido como referencia.

a3) Insertar un elemento en la última posición de la lista:

Igual que en el caso anterior, partiremos de una lista no vacía, y de nuevo para simplificar, que Lista está apuntando al último elemento de la lista:

El proceso es el siguiente:

`nodo->siguiente` debe apuntar a Lista->siguiente (NULL).
`Lista->siguiente` debe apuntar a `nodo`.
`nodo->anterior` apuntará a `Lista`.

a4) Insertar un elemento a continuación de un nodo cualquiera de una lista:

Partimos de una lista no vacía, e insertaremos un nodo a continuación de uno nodo cualquiera que no sea el último de la lista:

El proceso sigue siendo muy sencillo:

1) Hacemos que `nodo->siguiente` apunte a `lista->siguiente`
2) Hacemos que Lista->siguiente apunte a `nodo`.
3) Hacemos que `nodo->anterior` apunte a `lista`.
4) Hacemos que `nodo->siguiente->anterior` apunte a `nodo`.

Lo que hemos hecho es trabajar como si tuviéramos dos listas enlazadas, los dos primeros pasos equivalen a lo que hacíamos para insertar elementos en una lista abierta corriente.

Los dos siguientes pasos hacen lo mismo con la lista que enlaza los nodos en sentido contrario.

El paso 4 es el más oscuro, quizás requiera alguna explicación.

Supongamos que disponemos de un puntero auxiliar, "`p`" y que antes de empezar a insertar nodo, hacemos que apunte al nodo que quedará a continuación de nodo después de insertarlo, es decir

```
p = Lista->siguiente
```

Ahora empezamos el proceso de inserción, ejecutamos los pasos 1, 2 y 3. El cuarto sería sólo hacer que `p->anterior` apunte a nodo. Pero `nodo->siguiente` ya apunta a `p`, así que en realidad no necesitamos el puntero auxiliar, bastará con hacer que `nodo->siguiente->anterior` apunte a nodo.

a5) Añadir elemento en una lista doblemente enlazada, caso general:

Para generalizar todos los casos anteriores, sólo necesitamos añadir una operación:

Si lista está vacía hacemos que `Lista` apunte a `nodo`. Y `nodo->anterior` y `nodo->siguiente` a NULL.
Si lista no está vacía, hacemos que `nodo->siguiente` apunte a `Lista->siguiente`.

Después que `Lista->siguiente` apunte a `nodo`.
Hacemos que `nodo->anterior` apunte a `Lista`.

Si `nodo->siguiente` no es NULL, entonces hacemos que `nodo->siguiente->anterior` apunte a `nodo`
.

El paso 1 es equivalente a insertar un nodo en una lista vacía.

Los pasos 2 y 3 equivalen a la inserción en una lista enlazada corriente.

Los pasos 4, 5 equivalen a insertar en una lista que recorre los nodos en sentido contrario.

Existen más casos, las listas doblemente enlazadas son mucho más versátiles, pero todos los casos pueden reducirse a uno de los que hemos explicado aquí.

b) Buscar un elemento en una lista doblemente enlazada:

En muchos aspectos, una lista doblemente enlazada se comporta como dos listas abiertas que comparten los datos. En ese sentido, todo lo dicho en el capítulo sobre la **localización en listas enlazadas** se puede aplicar a listas doblemente enlazadas.

Pero además tenemos la ventaja de que podemos avanzar y retroceder desde cualquier nodo, sin necesidad de volver a uno de los extremos de la lista.

Por supuesto, se pueden hacer listas doblemente enlazadas no ordenadas, existen cientos de problemas que pueden requerir de este tipo de estructuras. Pero parece que la aplicación más sencilla de listas doblemente enlazadas es hacer arrays dinámicos ordenados, donde buscar un elemento concreto a partir de cualquier otro es más sencillo que en una lista abierta corriente.

Para recorrer una lista procederemos de un modo parecido al que usábamos con las listas abiertas, ahora no necesitamos un puntero auxiliar, pero tenemos que tener en cuenta que Lista no tiene por qué estar en uno de los extremos:

Retrocedemos hasta el comienzo de la lista, asignamos a lista el valor de `lista->anterior` mientras `lista->anterior` no sea NULL.
Abriremos un bucle que al menos debe tener una condición, que el índice no sea NULL.
Dentro del bucle asignaremos a lista el valor del nodo siguiente al actual.

Por ejemplo, para mostrar todos los valores de los nodos de una lista, podemos usar el siguiente bucle en C:

```
typedef struct _nodo {
  int dato;
  struct _nodo *siguiente;
  struct _nodo *anterior;
  } tipoNodo;

  typedef tipoNodo *pNodo;
  typedef tipoNodo *Lista;
  ...
  pNodo = indice;
  ...
  indice = Lista;
  while(indice->anterior) indice = indice->anterior;
  while(indice) {
     printf("%d\n", indice->dato);
     indice = indice->siguiente;
  }
  ...
```

Es importante que no perdamos el nodo `Lista`, si por error le asignáramos un valor de un puntero a un nodo que no esté en la lista, no podríamos acceder de nuevo a ella.

Es por eso que tendremos especial cuidado en no asignar el valor NULL a Lista.

b)Eliminar un nodo de una lista doblemente enlazada:

Analizaremos casos diferentes:

c1) Eliminar el único nodo de una lista doblemente enlazada.
c2) Eliminar el primer nodo.
c3) Eliminar el último nodo.
c4) Eliminar un nodo intermedio.

Para los casos que lo permitan consideraremos dos casos: que el nodo a eliminar es el actualmente apuntado por Lista o que no.

c1) Eliminar el único nodo en una lista doblemente enlazada:

En este caso, ese nodo será el apuntado por Lista.

Eliminamos el nodo.
Hacemos que Lista apunte a NULL.

c2) Eliminar el primer nodo de una lista doblemente enlazada:

Tenemos los dos casos posibles, que el nodo a borrar esté apuntado por Lista o que no. Si lo está, simplemente hacemos que `Lista` sea `Lista->siguiente`.

Si nodo apunta a Lista, hacemos que `Lista` apunte a `Lista->siguiente`.
Hacemos que `nodo->siguiente->anterior` apunte a `NULL`
Borramos el nodo apuntado por nodo.

El paso 2 separa el nodo a borrar del resto de la lista, independientemente del nodo al que apunte Lista.

c3) Eliminar el último nodo de una lista doblemente enlazada:

De nuevo tenemos los dos casos posibles, que el nodo a borrar esté apuntado por Lista o que no. Si lo está, simplemente hacemos que Lista sea Lista->anterior.

Si `nodo` apunta a `Lista`, hacemos que `Lista` apunte a `Lista->anterior`.
Hacemos que `nodo->anterior->siguiente` apunte a `NULL`
Borramos el nodo apuntado por nodo.

El paso 2 depara el nodo a borrar del resto de la lista, independientemente del nodo al que apunte Lista.

c4) Eliminar un nodo intermedio de una lista doblemente enlazada:

De nuevo tenemos los dos casos posibles, que el nodo a borrar esté apuntado por Lista o que no. Si lo está, simplemente hacemos que Lista sea Lista->anterior o Lista->siguiente

Se trata de un caso más general de los dos casos anteriores..

Si nodo apunta a Lista, hacemos que `Lista` apunte a `Lista->anterior` (o Lista->siguiente).
Hacemos que `nodo->anterior->siguiente` apunte a `nodo->siguiente`.
Hacemos que `nodo->siguiente->anterior` apunte a `nodo->anterior`.
Borramos el nodo apuntado por nodo.

c5) Eliminar un nodo de una lista doblemente enlazada, caso general:

De nuevo tenemos los dos casos posibles, que el nodo a borrar esté apuntado por Lista o que no. Si lo está, simplemente hacemos que Lista sea Lista->anterior, si no es NULL o Lista->siguiente en caso contrario.

Si nodo apunta a Lista,
Si `Lista->anterior` no es `NULL` hacemos que `Lista` apunte a `Lista->anterior`.
Si `Lista->siguiente` no es `NULL` hacemos que `Lista` apunte a `Lista->siguiente`.
Si ambos son `NULL`, hacemos que `Lista` sea `NULL`.
Si `nodo->anterior` no es `NULL`, hacemos que `nodo->anterior->siguiente` apunte a `nodo->siguiente`.

Si `nodo->siguiente` no es `NULL`, hacemos que `nodo->siguiente->anterior` apunte a `nodo->anterior`.
Borramos el nodo apuntado por nodo.

Ejemplo de lista doblemente enlazada en C:

Como en el caso de los ejemplos anteriores, construiremos una lista doblemente enlazada para almacenar números enteros. Para aprovechar mejor las posibilidades de estas listas, haremos que la lista esté ordenada. Haremos pruebas insertando varios valores, buscándolos y eliminándolos alternativamente para comprobar el resultado.

Algoritmo de inserción:

El primer paso es crear un nodo para el dato que vamos a insertar.

Si Lista está vacía, o el valor del primer elemento de la lista es mayor que el del nuevo, insertaremos el nuevo nodo en la primera posición de la lista.

En caso contrario, buscaremos el lugar adecuado para la inserción, tenemos un puntero "anterior". Lo inicializamos con el valor de Lista, y avanzaremos mientras anterior->siguiente no sea NULL y el dato que contiene anterior->siguiente sea menor o igual que el dato que queremos insertar.

Ahora ya tenemos anterior señalando al nodo adecuado, así que insertamos el nuevo nodo a continuación de él.

```
void Insertar(Lista *lista, int v)
{
   pNodo nuevo, actual;

   /* Crear un nodo nuevo */
   nuevo = (pNodo)malloc(sizeof(tipoNodo));
   nuevo->valor = v;

   /* Colocamos actual en la primera posición de la lista */
   actual = *lista;
   if(actual) while(actual->anterior) actual = actual->anterior;

   /* Si la lista está vacía o el primer miembro es mayor que el nuevo */
   if(!actual || actual->valor > v) {
      /* Añadimos la lista a continuación del nuevo nodo */
      nuevo->siguiente = actual;
      nuevo->anterior = NULL;
      if(actual) actual->anterior = nuevo;
      if(!*lista) *lista = nuevo;
   }
   else {
      /* Avanzamos hasta el último elemento o hasta que el siguiente tenga
         un valor mayor que v */
      while(actual->siguiente &&actual->siguiente->valor <= v)
```

```
         actual = actual->siguiente;
      /* Insertamos el nuevo nodo después del nodo anterior */
      nuevo->siguiente = actual->siguiente;
      actual->siguiente = nuevo;
      nuevo->anterior = actual;
      if(nuevo->siguiente) nuevo->siguiente->anterior = nuevo;
   }
}
```

Algoritmo de la función "Borrar":

Localizamos el nodo de valor v
¿Existe?
SI:
¿Es el nodo apuntado por lista?
SI: Hacer que lista apunte a otro sitio.
¿Es el primer nodo de la lista?
NO: nodo->anterior->siguiente = nodo->siguiente
¿Es el último nodo de la lista?
NO: nodo->siguiente->anterior = nodo->anterior
Borrar nodo

```
void Borrar(Lista *lista, int v)
{
   pNodo nodo;

   /* Buscar el nodo de valor v */
   nodo = *lista;
   while(nodo && nodo->valor <v) nodo = nodo->siguiente;
   while(nodo && nodo->valor > v) nodo = nodo->anterior;

   /* El valor v no está en la lista */
   if(!nodo || nodo->valor != v) return;

   /* Borrar el nodo */
   /* Si lista apunta al nodo que queremos borrar, apuntar a otro */
   if(nodo == *lista)
     if(nodo->anterior) *lista = nodo->anterior;
     else *lista = nodo->siguiente;

   if(nodo->anterior) /* no es el primer elemento */
      nodo->anterior->siguiente = nodo->siguiente;
   if(nodo->siguiente) /* no es el último nodo */
      nodo->siguiente->anterior = nodo->anterior;
   free(nodo);
}
```

Código del ejemplo completo:

```
#include <stdlib.h>
#include <stdio.h>

#define ASCENDENTE 1
#define DESCENDENTE 0

typedef struct _nodo {
   int valor;
   struct _nodo *siguiente;
   struct _nodo *anterior;
} tipoNodo;

typedef tipoNodo *pNodo;
typedef tipoNodo *Lista;

/* Funciones con listas: */
void Insertar(Lista *l, int v);
void Borrar(Lista *l, int v);

void BorrarLista(Lista *);
void MostrarLista(Lista l, int orden);

int main()
{
   Lista lista = NULL;
   pNodo p;

   Insertar(&lista, 20);
   Insertar(&lista, 10);
   Insertar(&lista, 40);
   Insertar(&lista, 30);

   MostrarLista(lista, ASCENDENTE);
   MostrarLista(lista, DESCENDENTE);

   Borrar(&lista, 10);
   Borrar(&lista, 15);
   Borrar(&lista, 45);
   Borrar(&lista, 30);

   MostrarLista(lista, ASCENDENTE);
   MostrarLista(lista, DESCENDENTE);

   BorrarLista(&lista);

   system("PAUSE");
   return 0;
}

void Insertar(Lista *lista, int v)
{
   pNodo nuevo, actual;

   /* Crear un nodo nuevo */
   nuevo = (pNodo)malloc(sizeof(tipoNodo));
```

```
   nuevo->valor = v;

   /* Colocamos actual en la primera posición de la lista */
   actual = *lista;
   if(actual) while(actual->anterior) actual = actual->anterior;
   /* Si la lista está vacía o el primer miembro es mayor que el nuevo */
   if(!actual || actual->valor > v) {
      /* Añadimos la lista a continuación del nuevo nodo */
      nuevo->siguiente = actual;
      nuevo->anterior = NULL;
      if(actual) actual->anterior = nuevo;
      if(!*lista) *lista = nuevo;
   }
   else {
      /* Avanzamos hasta el último elemento o hasta que el siguiente tenga
         un valor mayor que v */
      while(actual->siguiente &&actual->siguiente->valor <= v)
         actual = actual->siguiente;
      /* Insertamos el nuevo nodo después del nodo anterior */
      nuevo->siguiente = actual->siguiente;
      actual->siguiente = nuevo;
      nuevo->anterior = actual;
      if(nuevo->siguiente) nuevo->siguiente->anterior = nuevo;
   }
}

void Borrar(Lista *lista, int v)
{
   pNodo nodo;

   /* Buscar el nodo de valor v */
   nodo = *lista;
   while(nodo && nodo->valor < v) nodo = nodo->siguiente;
   while(nodo && nodo->valor > v) nodo = nodo->anterior;

   /* El valor v no está en la lista */
   if(!nodo || nodo->valor != v) return;

   /* Borrar el nodo */
   /* Si lista apunta al nodo que queremos borrar, apuntar a otro */
   if(nodo == *lista)
     if(nodo->anterior) *lista = nodo->anterior;
     else *lista = nodo->siguiente;

   if(nodo->anterior) /* no es el primer elemento */
      nodo->anterior->siguiente = nodo->siguiente;
   if(nodo->siguiente) /* no es el último nodo */
      nodo->siguiente->anterior = nodo->anterior;
   free(nodo);
}

void BorrarLista(Lista *lista)
{
   pNodo nodo, actual;

   actual = *lista;
   while(actual->anterior) actual = actual->anterior;
```

```
   while(actual) {
      nodo = actual;
      actual = actual->siguiente;
      free(nodo);
   }
   *lista = NULL;
}

void MostrarLista(Lista lista, int orden)
{
   pNodo nodo = lista;

   if(!lista) printf("Lista vacía");

   nodo = lista;
   if(orden == ASCENDENTE) {
      while(nodo->anterior) nodo = nodo->anterior;
      printf("Orden ascendente: ");
      while(nodo) {
         printf("%d -> ", nodo->valor);
         nodo = nodo->siguiente;
      }
   }
   else {
      while(nodo->siguiente) nodo = nodo->siguiente;
      printf("Orden descendente: ");
      while(nodo) {
         printf("%d -> ", nodo->valor);
         nodo = nodo->anterior;
      }
   }

   printf("\n");
}
```

Árboles:

Definición:

Un **árbol** es una **estructura no lineal** en la que **cada nodo puede apuntar a uno o varios nodos**.

También se suele dar una definición **recursiva**: un **árbol** es una estructura en compuesta por un dato y varios **árboles**.

Si un árbol no está vacío, el primer nodo recibe el nombre de **raíz.**

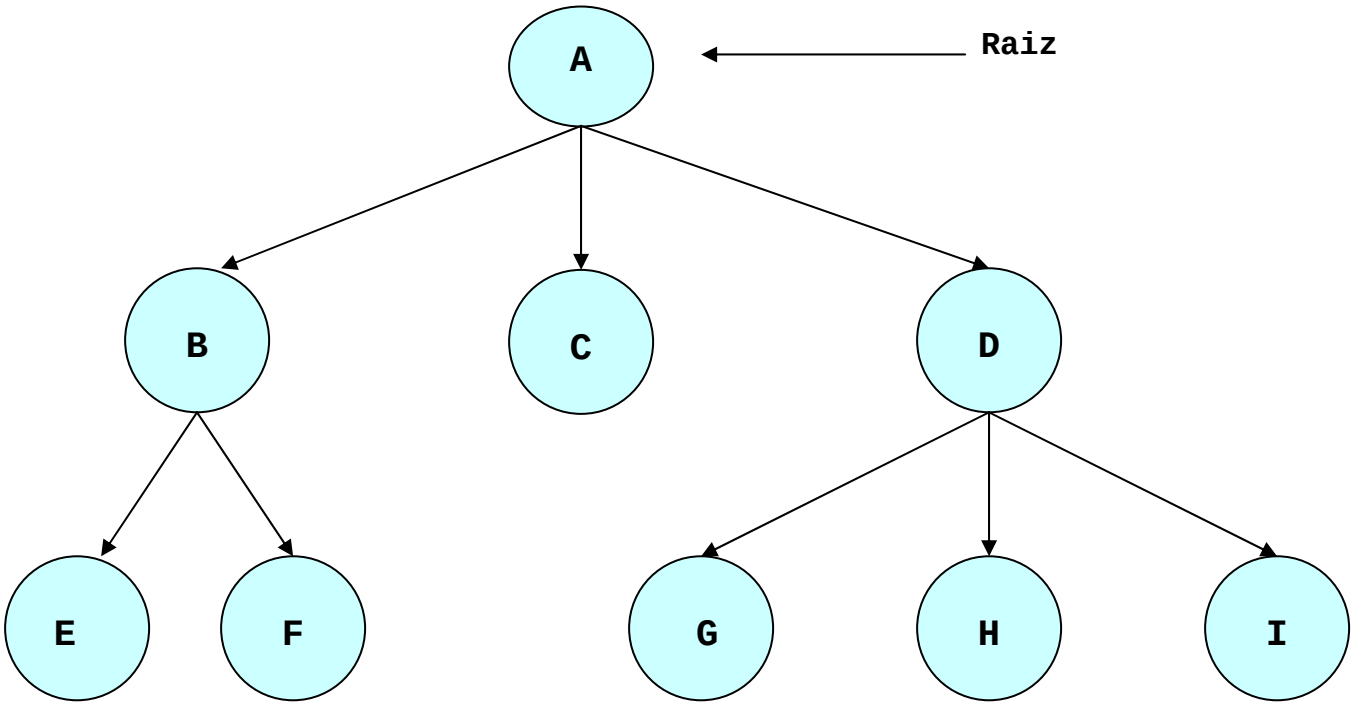


Figura 1

Un nodo es considerado como padre si tiene nodos sucesores. Los nodos sucesores se llaman **hijos**. Por ejemplo, el nodo B es el padre de los nodos hijos E y F. El padre de H es D. Son padres A, B, D.

Dos o más nodos con el mismo padre, se llaman **hermanos**. Los nodos G, h, I son hermanos.

Un nodo sin hijos se llama **hoja** (nodo terminal). Los nodos C, E, F, G, H, I son hojas.

Los hijos de un nodo y los hijos de este se llaman **descendientes**. El padre y el abuelo de un nodo son sus **ascendientes**.

Un **camino** es una secuencia de nodos. Cada nodo puede ser alcanzado por un único camino que empieza en el nodo raíz. La **longitud** de un camino es el número de pasos (punteros) entre nodos. La **longitud** del camino ADI es 2.

El **nivel** de un nodo es su distancia de la raiz. El raíz es de nivel cero. Los nodos hijos del raíz (B, C, D) son de **nivel** 1. El anterior **árbol** es de **nivel** 2.

La **altura** del árbol es el nivel más alto aumentado en 1. El anterior es un árbol de nivel 3

Orden: es el número potencial de hijos que puede tener cada elemento de árbol. De este modo, diremos que un árbol en el que cada nodo puede apuntar a otros dos es de *orden dos*, si puede apuntar a tres será de *orden tres*, etc.

Grado: el número de hijos que tiene el elemento con más hijos dentro del árbol. En el árbol del ejemplo, el **grado** es tres, ya que D tiene tres hijos, y no existen elementos con más de tres hijos. Los árboles de grado dos reciben el nombre de **árboles binarios.**

Las fechas de los gráficos representan **punteros** y se llaman **ramas.**

Un árbol se divide en **subárboles.** Un **subárbol** es cualquier estructura conectada por debajo del ríz**.** Cada **nodo** de un árbol es la raíz de un **subárbol**

Los árboles con los que trabajaremos tienen otra característica importante: cada **nodo sólo puede ser apuntado por otro nodo**, es decir, **cada nodo sólo tendrá un padre**. Esto hace que estos árboles estén fuertemente jerarquizados, y es lo que en realidad les da la apariencia de árboles.

Otra característica que normalmente tendrán nuestros árboles es que todos los nodos contengan el mismo número de punteros, es decir, usaremos la misma estructura para todos los nodos del árbol. Esto hace que la estructura sea más sencilla, y por lo tanto también los programas para trabajar con ellos.

Tampoco es necesario que todos los nodos hijos de un nodo concreto existan. Es decir, que pueden usarse todos, algunos o ninguno de los punteros de cada nodo.

Un árbol en el que en cada nodo o bien todos o ninguno de los hijos existe, se llama **árbol completo**.

En una cosa, los árboles se parecen al resto de las estructuras que hemos visto: dado un nodo cualquiera de la estructura, podemos considerarlo como una estructura independiente. Es decir, un nodo cualquiera puede ser considerado como la raíz de un árbol completo.

Existen otros conceptos que definen las características del árbol, en relación a su tamaño:

Los árboles de orden dos son bastante especiales, de hecho les dedicaremos varios capítulos. Estos árboles se conocen también como **árboles binarios**.

Frecuentemente, aunque tampoco es estrictamente necesario, para hacer más fácil moverse a través del árbol, añadiremos un puntero a cada nodo que apunte al nodo padre. De este modo podremos avanzar en dirección a la raíz, y no sólo hacia las hojas.

Es importante conservar siempre el nodo *raíz* ya que es el nodo a partir del cual se desarrolla el árbol, si perdemos este nodo, perderemos el acceso a todo el árbol.

El nodo típico de un árbol difiere de los nodos que hemos visto hasta ahora para listas, aunque sólo en el número de nodos. Veamos un ejemplo de nodo para crear árboles de orden tres:

```
struct nodo {
   int dato;
   struct nodo *rama1;
   struct nodo *rama2;
   struct nodo *rama3;
};
```

O generalizando más:

```
#define ORDEN 5

struct nodo {
   int dato;
   struct nodo *rama[ORDEN];
};
```

Declaraciones de tipos para manejar árboles en C:

Para C, y basándonos en la declaración de nodo que hemos visto, trabajaremos con los siguientes tipos:

```
typedef struct _nodo {
   int dato;
   struct _nodo *rama[ORDEN];
} tipoNodo;

typedef tipoNodo *pNodo;
typedef tipoNodo *Arbol;
```

Al igual que hicimos con las listas que hemos visto hasta ahora, declaramos un tipo `tipoNodo` para declarar nodos, y un tipo `pNodo` para es el tipo para declarar punteros a un nodo.

`Arbol` es el tipo para declarar árboles de orden `ORDEN`.

El movimiento a través de árboles, salvo que implementemos punteros al nodo padre, será siempre partiendo **del nodo raíz hacia un nodo hoja**. Cada vez que lleguemos a un nuevo nodo podremos optar por cualquiera de los nodos a los que apunta para avanzar al siguiente nodo.

En general, intentaremos que exista algún significado asociado a cada uno de los punteros dentro de cada nodo, los árboles que estamos viendo son abstractos, pero las aplicaciones no tienen por qué serlo. Un ejemplo de estructura en árbol es el sistema de directorios y ficheros de un sistema operativo. Aunque en

este caso se trata de árboles con nodos de dos tipos, nodos directotio y nodos fichero, podríamos considerar que los nodos hoja son ficheros y los nodos rama son directorios.

Otro ejemplo podría ser la tabla de contenido de un libro, por ejemplo de este mismo curso, dividido en capítulos, y cada uno de ellos en subcapítulos. Aunque el libro sea algo lineal, como una lista, en el que cada capítulo sigue al anterior, también es posible acceder a cualquier punto de él a través de la tabla de contenido.

También se suelen organizar en forma de árbol los organigramas de mando en empresas o en el ejército, y los árboles genealógicos.

Operaciones básicas con árboles:

Salvo que trabajemos con árboles especiales, las inserciones serán siempre en punteros de nodos hoja o en punteros libres de nodos rama. Con estas estructuras no es tan fácil generalizar, ya que existen muchas variedades de árboles.

De nuevo tenemos casi el mismo repertorio de operaciones de las que disponíamos con las listas:

a) Recorrer el árbol completo.
Añadir o insertar elementos.
Buscar o localizar elementos.
Borrar elementos.
Moverse a través del árbol.

Los algoritmos de inserción y borrado dependen en gran medida del tipo de árbol que estemos implementando, de modo que por ahora los pasaremos por alto y nos centraremos más en el modo de recorrer árboles.

a) Recorrer el árbol:

El modo evidente de moverse a través de las ramas de un árbol es siguiendo los punteros, del mismo modo en que nos movíamos a través de las listas.

Esos recorridos dependen en gran medida del tipo y propósito del árbol, pero hay ciertos recorridos que usaremos frecuentemente. Se trata de aquellos recorridos que incluyen todo el árbol.

Hay tres formas de recorrer un árbol completo, y las tres se suelen implementar mediante recursividad. En los tres casos se sigue siempre a partir de cada nodo todas las ramas una por una.

Supongamos que tenemos un árbol de orden tres, y queremos recorrerlo por completo.

Partiremos del nodo raíz:

```
RecorrerArbol(raiz);
```

La función `RecorrerArbol`, aplicando recursividad, será tan sencilla como invocar de nuevo a la función `RecorrerArbol` para cada una de las ramas:

```
void RecorrerArbol(arbol a)
 {
   if(a == NULL) return;
```

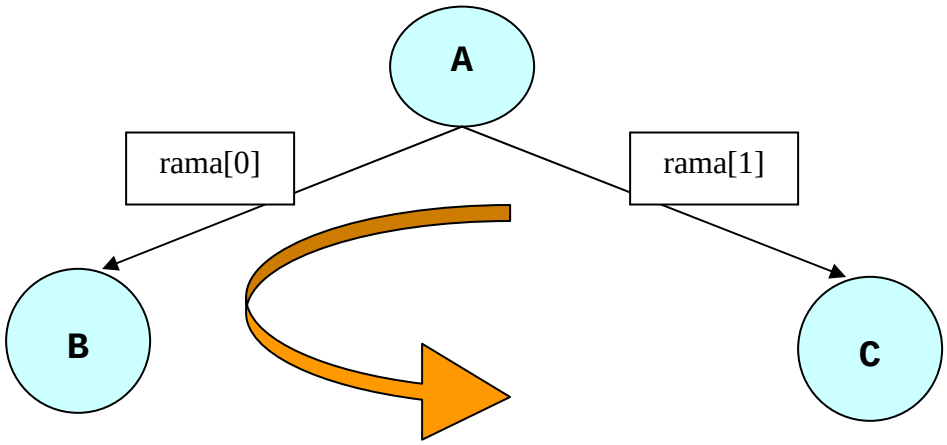
```
   RecorrerArbol(a->rama[0]);
   RecorrerArbol(a->rama[1]);
   RecorrerArbol(a->rama[2]);
 }
```

Lo que diferencia los distintos métodos de recorrer el árbol no es el sistema de hacerlo, sino el momento que elegimos para procesar el valor de cada nodo con relación a los recorridos de cada una de las ramas.

Los tres tipos son:

Pre-orden:

En este tipo de recorrido, el valor del nodo se procesa antes de recorrer las ramas:



El recorrido en PREORDEN es **A B C**

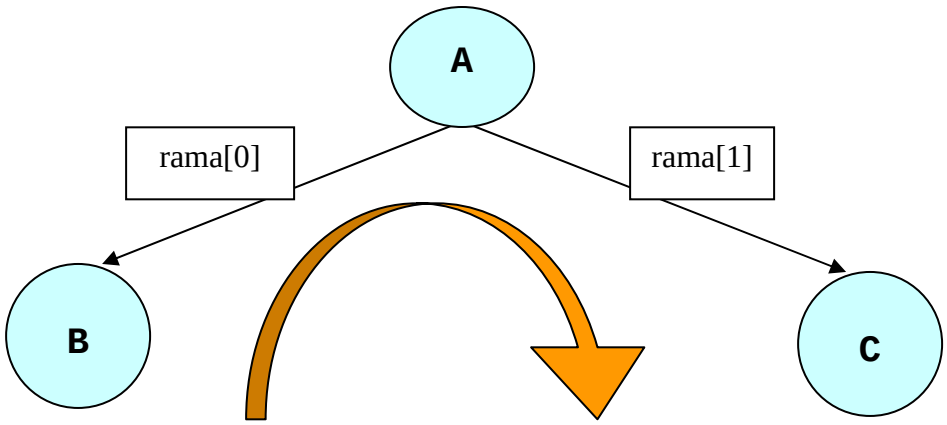
```
void PreOrden(arbol a)
{
   if(a == NULL) return;
   Procesar(dato);
   RecorrerArbol(a->rama[0]);
   RecorrerArbol(a->rama[1]);
  }
```

El recorrido en preorden del arbol de la **Figura 1** sería: A B E F C D G H I

In-orden:

En este tipo de recorrido, el valor del nodo se procesa después de recorrer la primera rama y antes de recorrer la última. Esto tiene más sentido en el caso de árboles binarios, y también cuando existen ORDEN-1 datos, en cuyo caso procesaremos cada dato entre el recorrido de cada dos ramas (este es el caso de los árboles-b):

El recorrido es: **B A C**

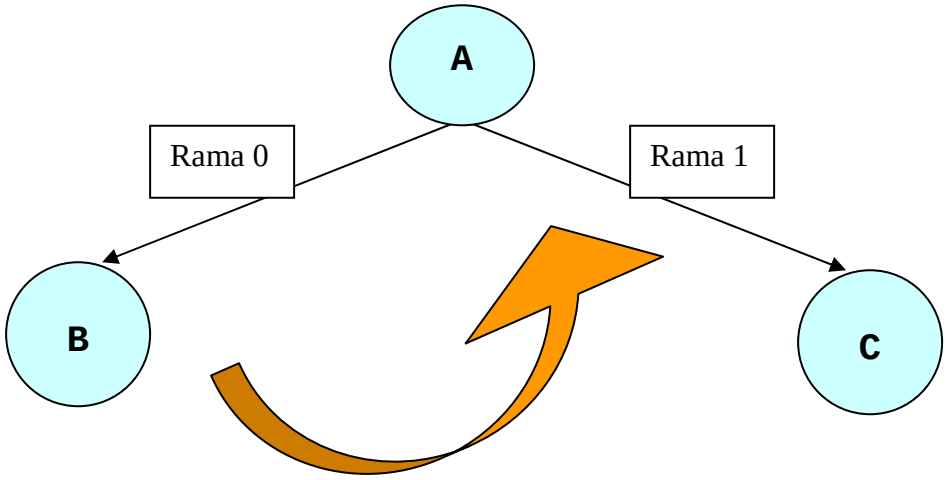



```
void InOrden(arbol a)
{
   if(a == NULL) return;
   RecorrerArbol(a->rama[0]);
   Procesar(dato);
   RecorrerArbol(a->rama[1]);
  }
```

El recorrido en IN-ORDEN del arbol de la **Figura 1** sería: **E B F A C G D H I**

Post-orden:

En este tipo de recorrido, el valor del nodo se procesa después de recorrer todas las ramas: **B C A**




```
void PostOrden(arbol a)
{
   if(a == NULL) return;
   RecorrerArbol(a->rama[0]);
   RecorrerArbol(a->rama[1]);
   RecorrerArbol(a->rama[2]);
   Procesar(dato);
}
```

El recorrido en pos-orden del arbol de la **Figura 1** sería: **E F B C G H I D A**

Eliminar nodos de un árbol:

El proceso general es muy sencillo en este caso, pero con una importante limitación, sólo podemos borrar nodos hoja:

El proceso sería el siguiente:

Buscar el nodo padre del que queremos eliminar.
Buscar el puntero del nodo padre que apunta al nodo que queremos borrar.
Liberar el nodo.
padre->nodo[i] = NULL;.

Cuando el nodo a borrar no sea un nodo hoja, diremos que hacemos una "poda", y en ese caso eliminaremos el árbol cuya raíz es el nodo a borrar. Se trata de un procedimiento recursivo, aplicamos el recorrido PostOrden, y el proceso será borrar el nodo.

El procedimiento es similar al de borrado de un nodo:

Buscar el nodo padre del que queremos eliminar.
Buscar el puntero del nodo padre que apunta al nodo que queremos borrar.
Podar el árbol cuyo padre es nodo.
padre->nodo[i] = NULL;.

Árboles ordenados:

Un árbol ordenado, en general, es aquel a partir del cual se puede obtener una secuencia ordenada siguiendo uno de los recorridos posibles del árbol: **inorden, preorden o postorden.**

En estos árboles es importante que la secuencia se mantenga ordenada aunque se añadan o se eliminen nodos.

Existen varios tipos de árboles ordenados, que veremos a continuación:

Árboles binarios de búsqueda (ABB):

Son árboles de orden 2 que mantienen una secuencia ordenada si se recorren en inorden.

Árboles AVL:

Son árboles binarios de búsqueda equilibrados, es decir, los niveles de cada rama para cualquier nodo no difieren en más de 1.

Árboles perfectamente equilibrados:

Son árboles binarios de búsqueda en los que el número de nodos de cada rama para cualquier nodo no difieren en más de 1. Son por lo tanto árboles AVL también.

Árboles 2-3:

Son árboles de orden 3, que contienen dos claves en cada nodo y que están también equilibrados. También generan secuencias ordenadas al recorrerlos en inorden.

Árboles-B:

Caso general de árboles 2-3, que para un orden M, contienen M-1 claves.

Árboles binarios de Búsqueda (ABB)

Se trata de árboles de orden 2 en los que se cumple que para cada nodo, el valor de la clave de la raíz del subárbol izquierdo es menor que el valor de la clave del nodo y que el valor de la clave raíz del subárbol derecho es mayor que el valor de la clave del nodo.

Operaciones en ABB

El repertorio de operaciones que se pueden realizar sobre un ABB es parecido al que realizábamos sobre otras estructuras de datos, más alguna otra propia de árboles:

Buscar un elemento.
Insertar un elemento.
Borrar un elemento.
Movimientos a través del árbol:
Izquierda.
Derecha.
Raiz.
Información:
Comprobar si un árbol está vacío.
Calcular el número de nodos.
Comprobar si el nodo es hoja.
Calcular la altura de un nodo.
Calcular la altura de un árbol.

Buscar un elemento:

Partiendo siempre del nodo raíz, el modo de buscar un elemento se define de forma **recursiva:**

Si el árbol está vacío, terminamos la búsqueda: el elemento no está en el árbol.
Si el valor del nodo raíz es igual que el del elemento que buscamos, terminamos la búsqueda con éxito.
Si el valor del nodo raíz es mayor que el elemento que buscamos, continuaremos la búsqueda en el árbol izquierdo.
Si el valor del nodo raíz es menor que el elemento que buscamos, continuaremos la búsqueda en el árbol derecho.

El valor de retorno de una función de búsqueda en un ABB puede ser un puntero al nodo encontrado, o NULL, si no se ha encontrado.

Insertar un elemento:

Para insertar un elemento nos basamos en el algoritmo de búsqueda. Si el elemento está en el árbol no lo insertaremos. Si no lo está, lo insertaremos a continuación del último nodo visitado.

Necesitamos un puntero auxiliar para conservar una referencia al padre del nodo raíz actual. El valor inicial para ese puntero es NULL.

Padre = NULL
nodo = Raiz
Bucle: mientras actual no sea un árbol vacío o hasta que se encuentre el elemento.
Si el valor del nodo raíz es mayor que el elemento que buscamos, continuaremos la búsqueda en el árbol izquierdo: Padre=nodo, nodo=nodo->izquierdo.

Si el valor del nodo raíz es menor que el elemento que buscamos, continuaremos la búsqueda en el árbol derecho: Padre=nodo, nodo=nodo->derecho.
Si nodo no es NULL, el elemento está en el árbol, por lo tanto salimos.
Si Padre es NULL, el árbol estaba vacío, por lo tanto, el nuevo árbol sólo contendrá el nuevo elemento, que será la raíz del árbol.
Si el elemento es menor que el Padre, entonces insertamos el nuevo elemento como un nuevo árbol izquierdo de Padre.
Si el elemento es mayor que el Padre, entonces insertamos el nuevo elemento como un nuevo árbol derecho de Padre.

Este modo de actuar asegura que el árbol sigue siendo ABB.

Borrar un elemento:

Para borrar un elemento también nos basamos en el algoritmo de búsqueda. Si el elemento no está en el árbol no lo podremos borrar. Si está, hay dos casos posibles:

a) Se trata de un nodo hoja: en ese caso lo borraremos directamente.

b) Se trata de un nodo rama: en ese caso no podemos eliminarlo, puesto que perderíamos todos los elementos del árbol de que el nodo actual es padre. En su lugar buscamos el nodo más a la izquierda del subárbol derecho, o el más a la derecha del subárbol izquierdo e intercambiamos sus valores. A continuación eliminamos el nodo hoja.

Necesitamos un puntero auxiliar para **conservar** una referencia al padre del nodo raíz actual. El valor inicial para ese puntero es NULL.

Padre = NULL
Si el árbol está vacío: el elemento no está en el árbol, por lo tanto salimos sin eliminar ningún elemento.
(1) Si el valor del nodo raíz es igual que el del elemento que buscamos, estamos ante uno de los siguientes casos:
El nodo raíz es un nodo hoja:
Si 'Padre' es NULL, el nodo raíz es el único del árbol, por lo tanto el puntero al árbol debe ser NULL.
Si raíz es la rama derecha de 'Padre', hacemos que esa rama apunte a NULL.
Si raíz es la rama izquierda de 'Padre', hacemos que esa rama apunte a NULL.
Eliminamos el nodo, y salimos.
El nodo no es un nodo hoja:
Buscamos el 'nodo' más a la izquierda del árbol derecho de raíz o el más a la derecha del árbol izquierdo. Hay que tener en cuenta que puede que sólo exista uno de esos árboles. Al mismo tiempo, actualizamos 'Padre' para que apunte al padre de 'nodo'.
Intercambiamos los elementos de los nodos raíz y 'nodo'.
Borramos el nodo 'nodo'. Esto significa volver a (1), ya que puede suceder que 'nodo' no sea un nodo hoja. (Ver ejemplo 3)
Si el valor del nodo raíz es mayor que el elemento que buscamos, continuaremos la búsqueda en el árbol izquierdo.
Si el valor del nodo raíz es menor que el elemento que buscamos, continuaremos la búsqueda en el árbol derecho.

Ejemplo 1: Borrar un nodo hoja

Localizamos el nodo a borrar, al tiempo que mantenemos un puntero a 'Padre'.
Hacemos que el puntero de 'Padre' que apuntaba a 'nodo', ahora apunte a NULL.
Borramos el 'nodo'.

Ejemplo 2: Borrar un nodo rama con intercambio de un nodo hoja.

Localizamos el nodo a borrar ('raíz').
Buscamos el nodo más a la derecha del árbol izquierdo de 'raíz', en este caso el 3, al tiempo que mantenemos un puntero a 'Padre' a 'nodo'.
Intercambiamos los elementos 3 y 4.
Hacemos que el puntero de 'Padre' que apuntaba a 'nodo', ahora apunte a NULL.
Borramos el 'nodo'.

Ejemplo 3: Borrar un nodo rama con intercambio de un nodo rama.

Localizamos el nodo a borrar ('raíz').
Buscamos el nodo más a la izquierda del árbol derecho de 'raíz', en este caso el 12, ya que el árbol derecho no tiene nodos a su izquierda, si optamos por la rama izquierda, estaremos en un caso análogo.
Al mismo tiempo que mantenemos un puntero a 'Padre' a 'nodo'.
Intercambiamos los elementos 6 y 12.
Ahora tenemos que repetir el bucle para el nodo 6 de nuevo, ya que no podemos eliminarlo.

Localizamos de nuevo el nodo a borrar ('raíz').
Buscamos el nodo más a la izquierda del árbol derecho de 'raíz', en este caso el 16, al mismo tiempo que mantenemos un puntero a 'Padre' a 'nodo'.
Intercambiamos los elementos 6 y 16.
Hacemos que el puntero de 'Padre' que apuntaba a 'nodo', ahora apunte a NULL.
Borramos el 'nodo'.

Este modo de actuar asegura que el árbol sigue siendo ABB.

Movimientos a través del árbol:

No hay mucho que contar. Nuestra estructura se referenciará siempre mediante un puntero al nodo Raiz, este puntero no debe perderse nunca.

Para movernos a través del árbol usaremos punteros auxiliares, de modo que desde cualquier puntero los movimientos posibles serán: moverse al nodo raíz de la rama izquierda, moverse al nodo raíz de la rama derecha o moverse al nodo Raiz del árbol.

Información sobre un árbol:

Hay varios parámetros que podemos calcular o medir dentro de un árbol. Algunos de ellos nos darán idea de lo eficientemente que está organizado o el modo en que funciona.

Comprobar si un árbol está vacío.

Un árbol está vacío si su raíz es NULL.

Calcular el número de nodos.

Tenemos dos opciones para hacer esto, una es llevar siempre la cuenta de nodos en el árbol al mismo tiempo que se añaden o eliminan elementos. La otra es, sencillamente, contarlos.

Para contar los nodos podemos recurrir a cualquiera de los tres modos de recorrer el árbol: inorden, preorden o postorden, como acción sencillamente incrementamos el contador.

Comprobar si el nodo es hoja.

Esto es muy sencillo, basta con comprobar si tanto el árbol izquierdo como el derecho están vacíos. Si ambos lo están, se trata de un nodo hoja.

Calcular la altura de un nodo.

No hay un modo directo de hacer esto, ya que no nos es posible recorrer el árbol en la dirección de la raíz. De modo que tendremos que recurrir a otra técnica para calcular la altura.

Lo que haremos es buscar el elemento del nodo de que queremos averiguar la altura. Cada vez que avancemos un nodo incrementamos la variable que contendrá la altura del nodo.

Empezamos con el nodo raíz apuntando a Raiz, y la 'Altura' igual a cero.
Si el valor del nodo raíz es igual que el del elemento que buscamos, terminamos la búsqueda y el valor de la altura es 'Altura'.
Incrementamos 'Altura'.
Si el valor del nodo raíz es mayor que el elemento que buscamos, continuaremos la búsqueda en el árbol izquierdo.
Si el valor del nodo raíz es menor que el elemento que buscamos, continuaremos la búsqueda en el árbol derecho.

Calcular la altura de un árbol.

La altura del árbol es la altura del nodo de mayor altura. Para buscar este valor tendremos que recorrer todo el árbol, de nuevo es indiferente el tipo de recorrido que hagamos, cada vez que cambiemos de nivel incrementamos la variable que contiene la altura del nodo actual, cuando lleguemos a un nodo hoja compararemos su altura con la variable que contiene la altura del árbol si es mayor, actualizamos la altura del árbol.

Iniciamos un recorrido del árbol en postorden, con la variable de altura igual a cero.
Cada vez que empecemos a recorrer una nueva rama, incrementamos la altura para ese nodo.
Después de procesar las dos ramas, verificamos si la altura del nodo es mayor que la variable que almacena la altura actual del árbol, si es así, actualizamos esa variable.

Árboles degenerados:

Los árboles binarios de búsqueda tienen un gran inconveniente. Por ejemplo, supongamos que creamos un ABB a partir de una lista de valores ordenada:

2, 4, 5, 8, 9, 12

Difícilmente podremos llamar a la estructura resultante un árbol:

Esto es lo que llamamos un árbol binario de búsqueda **degenerado**.

Más adelante veremos una nueva estructura, el árbol AVL, que resuelve este problema, generando árboles de búsqueda equilibrados.

Ejemplo de ABB en lenguaje C:

Vamos a ver cómo implementar en C algunas de las funciones que hemos explicado para árboles ABB, al final se incluye un ejemplo completo para árboles de enteros.

Declaración de tipos:

Como estamos trabajando con un árbol binario, sólo necesitamos una estructura para referirnos tanto a cualquiera de los nodos como al árbol completo. Recuerda que cualquier nodo puede ser considerado como la raíz de un árbol.

```
typedef struct _nodo {
   int dato;
   struct _nodo *derecho;
   struct _nodo *izquierdo;
} tipoNodo;
```

```
typedef tipoNodo *pNodo;
typedef tipoNodo *Arbol;
```

Insertar un elemento en un árbol ABB:

Diseñaremos una función que se ajuste al algoritmo:

Padre = NULL
nodo = Raiz
Bucle: mientras actual no sea un árbol vacío o hasta que se encuentre el elemento.
Si el valor del nodo raíz es mayor que el elemento que buscamos, continuaremos la búsqueda en el árbol izquierdo: Padre=nodo, nodo=nodo->izquierdo.
Si el valor del nodo raíz es menor que el elemento que buscamos, continuaremos la búsqueda en el árbol derecho: Padre=nodo, nodo=nodo->derecho.
Si nodo no es NULL, el elemento está en el árbol, por lo tanto salimos.
Si Padre es NULL, el árbol estaba vacío, por lo tanto, el nuevo árbol sólo contendrá el nuevo elemento, que será la raíz del árbol.
Si el elemento es menor que el Padre, entonces insertamos el nuevo elemento como un nuevo árbol izquierdo de Padre.
Si el elemento es mayor que el Padre, entonces insertamos el nuevo elemento como un nuevo árbol derecho de Padre.

```
void Insertar(Arbol *a, int dat)
{
   pNodo padre = NULL; /* (1) */
   pNodo actual = *a;  /* (2) */

   while(!Vacio(actual) && dat != actual->dato) {  /* (3) */
      padre = actual;
      if(dat < actual->dato) actual = actual->izquierdo;  /* (3-a) */
      else if(dat > actual->dato) actual = actual->derecho; /* (3-b) */
   }

   if(!Vacio(actual)) return;  /* (4) */
   if(Vacio(padre)) {  /* (5) */
      *a = (Arbol)malloc(sizeof(tipoNodo));
      (*a)->dato = dat;
      (*a)->izquierdo = (*a)->derecho = NULL;
   }
   else if(dat < padre->dato) { /* (6) */
      actual = (Arbol)malloc(sizeof(tipoNodo));
      padre->izquierdo = actual;
      actual->dato = dat;
      actual->izquierdo = actual->derecho = NULL;
   }
   else if(dat > padre->dato) { /* (7) */
      actual = (Arbol)malloc(sizeof(tipoNodo));
      padre->derecho = actual;
      actual->dato = dat;
      actual->izquierdo = actual->derecho = NULL;
  }
}
```

Eliminar un elemento de un árbol ABB:

Diseñaremos una función que se ajuste al algoritmo:

Padre = NULL
Si el árbol está vacío: el elemento no está en el árbol, por lo tanto salimos sin eliminar ningún elemento.
Si el valor del nodo raíz es igual que el del elemento que buscamos, estamos ante uno de los siguientes casos:
El nodo raíz es un nodo hoja:
Si 'Padre' es NULL, el nodo raíz es el único del árbol, por lo tanto el puntero al árbol debe ser NULL.
Si raíz es la rama derecha de 'Padre', hacemos que esa rama apunte a NULL.
Si raíz es la rama izquierda de 'Padre', hacemos que esa rama apunte a NULL.
Eliminamos el nodo, y salimos.
El nodo no es un nodo hoja:
Buscamos el 'nodo' más a la izquierda del árbol derecho de raíz o el más a la derecha del árbol izquierdo. Hay que tener en cuenta que puede que sólo exista uno de esos árboles. Al mismo tiempo, actualizamos 'Padre' para que apunte al padre de 'nodo'.
Intercambiamos los elementos de los nodos raíz y 'nodo'.
Borramos el nodo 'nodo'. Esto significa volver a (3), ya que puede suceder que 'nodo' no sea un nodo hoja.
Si el valor del nodo raíz es mayor que el elemento que buscamos, continuaremos la búsqueda en el árbol izquierdo.
Si el valor del nodo raíz es menor que el elemento que buscamos, continuaremos la búsqueda en el árbol derecho.

```
void Borrar(Arbol *a, int dat)
{
   pNodo padre = NULL; /* (1) */
   pNodo actual;
   pNodo nodo;
   int aux;

   actual = *a;
   while(!Vacio(actual)) { /* Búsqueda */
      if(dat == actual->dato) {
         if(EsHoja(actual)) {
            if(padre)
               if(padre->derecho == actual) padre->derecho = NULL;
                else if(padre->izquierdo == actual) padre->izquierdo = NULL;
            free(actual);
            actual = NULL;
            return;
         }
         else {
            /* Buscar nodo */
            padre = actual;
            if(actual->derecho) {
               nodo = actual->derecho;
               while(nodo->izquierdo) {
                  padre = nodo;
                  nodo = nodo->izquierdo;
```

```
               }
            }
            else {
               nodo = actual->izquierdo;
               while(nodo->derecho) {
                  padre = nodo;
                  nodo = nodo->derecho;
               }
            }
            /* Intercambio */
            aux = actual->dato;
            actual->dato = nodo->dato;
            nodo->dato = aux;
            actual = nodo;
         }
      }
      else {
         padre = actual;
         if(dat > actual->dato) actual = actual->derecho;
         else if(dat < actual->dato) actual = actual->izquierdo;
      }
   }
}
```

Buscar un elemento en un árbol ABB:

Diseñaremos una función que se ajuste al algoritmo:

Si el árbol está vacío, terminamos la búsqueda: el elemento no está en el árbol.
Si el valor del nodo raíz es igual que el del elemento que buscamos, terminamos la búsqueda con éxito.
Si el valor del nodo raíz es mayor que el elemento que buscamos, continuaremos la búsqueda en el árbol izquierdo.
Si el valor del nodo raíz es menor que el elemento que buscamos, continuaremos la búsqueda en el árbol derecho.

```
int B uscar(Arbol a, int dat)
{
   pNodo actual = a;

   while(!Vacio(actual)) {
      if(dat == actual->dato) return TRUE; /* dato encontrado  (2) */
      else if(dat < actual->dato) actual = actual->izquierdo;  /* (3) */
      else if(dat > actual->dato) actual = actual->derecho; /* (4) */
   }
   return FALSE; /* No está en árbol (1) */
}
```

Comprobar si el árbol está vacío:

Esta función es fácil de implementar:

```
int Vacio(Arbol r)
{
   return r==NULL;
}
```

Comprobar si un nodo es hoja:

Esta función también es sencilla de implementar:

```
int EsHoja(pNodo r)
{
   return !r->derecho && !r->izquierdo;
}
```

Contar número de nodos:

Comentábamos que para contar los nodos podemos recurrir a cualquiera de los tres modos de recorrer el árbol: inorden, preorden o postorden y como acción incrementamos el contador de nodos. Para implementar este algoritmo recurrimos a dos funciones:

```
int NumeroNodos(Arbol a, int *contador)
{
   *contador = 0;

   auxContador(a, contador);
   return *contador;
}

void auxContador(Arbol nodo, int *c)
{
   (*c)++; /* Acción: incrementar número de nodos. (Preorden) */
   if(nodo->izquierdo) auxContador(nodo->izquierdo, c); /* Rama izquierda */
   if(nodo->derecho)   auxContador(nodo->derecho, c);   /* Rama derecha */
}
```

Calcular la altura de un árbol:

Es un problema parecido al anterior, pero ahora tenemos que contar la altura, no en número de nodos. Cada vez que lleguemos a un nodo hoja, verificamos si la altura del nodo es la máxima, y si lo es, actualizamos la altura del árbol a ese valor:

Iniciamos un recorrido del árbol en postorden, con la variable de altura igual a cero.
Cada vez que empecemos a recorrer una nueva rama, incrementamos la altura para ese nodo.

Después de procesar las dos ramas, verificamos si la altura del nodo es mayor que la variable que almacena la altura actual del árbol, si es así, actualizamos esa variable.

```
int AlturaArbol(Arbol a, int *altura)
{
   *altura = 0; /* (1) */

   auxAltura(a, 0, altura);
   return *altura;
}

void auxAltura(pNodo nodo, int a, int *altura)
{
   /* (2) Cada vez que llamamos a auxAltura pasamos como parámetro a+1 */
   if(nodo->izquierdo) auxAltura(nodo->izquierdo, a+1, altura); /* Rama izquierda */
   if(nodo->derecho)   auxAltura(nodo->derecho, a+1, altura);   /* Rama derecha */
   if(EsHoja(nodo) && a > *altura) *altura = a; /* Proceso (Postorden)  (3) */
}
```

Calcular la altura del nodo que contiene un dato concreto:

Lo que haremos será buscar el elemento del nodo del que queremos averiguar la altura. Cada vez que avancemos un nodo incrementamos la variable que contendrá la altura del nodo.

Empezamos con el nodo raíz apuntando a Raiz, y la 'Altura' igual a cero.
Si el valor del nodo raíz es igual que el del elemento que buscamos, terminamos la búsqueda y el valor de la altura es 'Altura'.
Incrementamos 'Altura'.
Si el valor del nodo raíz es mayor que el elemento que buscamos, continuaremos la búsqueda en el árbol izquierdo.
Si el valor del nodo raíz es menor que el elemento que buscamos, continuaremos la búsqueda en el árbol derecho.

```
int A ltura(Arbol a, int dat)
{
   int altura = 0;
   pNodo actual = a; /* (1) */

   while(!Vacio(actual)) {
      if(dat == actual->dato) return altura; /* dato encontrado. (2) */
      else {
         altura++; /* (3) */
         if(dat < actual->dato) actual = actual->izquierdo; /* (4) */
         else if(dat > actual->dato) actual = actual->derecho; /* (5) */
      }
   }
   return -1; /* No está en árbol */
}
```

Aplicar una función a cada elemento del árbol, según los tres posibles recorridos:

Todos los recorridos se aplican de forma recursiva. En este ejemplo crearemos tres funciones, una por cada tipo de recorrido, que aplicarán una función al elemento de cada nodo.

La función a aplicar puede ser cualquiera que admita como parámetro un puntero a un entero, y que no tenga valor de retorno.

InOrden:

En Inorden, primero procesamos el subárbol izquierdo, después el nodo actual, y finalmente el subárbol derecho:

```
void InOrden(Arbol a, void (*func)(int*))
{
   if(a->izquierdo) InOrden(a->izquierdo, func); /* Subárbol izquierdo */
   func(&(a->dato)); /* Aplicar la función al dato del nodo actual */
   if(a->derecho) InOrden(a->derecho, func);     /* Subárbol derecho */
}
```

PreOrden:

En Preorden, primero procesamos el nodo actual, después el subárbol izquierdo, y finalmente el subárbol derecho:

```
void PreOrden(Arbol a, void (*func)(int*))
{
   func(&(a->dato)); /* Aplicar la función al dato del nodo actual */
   if(a->izquierdo) PreOrden(a->izquierdo, func); /* Subárbol izquierdo */
   if(a->derecho) PreOrden(a->derecho, func);     /* Subárbol derecho */
}
```

PostOrden:

En Postorden, primero procesamos el subárbol izquierdo, después el subárbol derecho, y finalmente el nodo actual:

```
void PostOrden(Arbol a, void (*func)(int*))
{
   if(a->izquierdo) PostOrden(a->izquierdo, func); /* Subárbol izquierdo */
   if(a->derecho) PostOrden(a->derecho, func);     /* Subárbol derecho */
   func(&(a->dato)); /* Aplicar la función al dato del nodo actual */
}
```

Ejemplo de ABB en Lenguaje C++:

Haremos ahora lo mismo que en el ejemplo en C, pero incluyendo todas las funciones y datos en una única clase.

Declaración de clase ArbolABB:

Declaramos dos clases, una para nodo y otra para ArbolABB, la clase nodo la declararemos como parte de la clase ArbolABB, de modo que no tendremos que definir relaciones de amistad, y evitamos que otras clases o funciones tengan acceso a los datos internos de nodo.

```
class ArbolABB {
  private:
   //// Clase local de Lista para Nodo de ArbolBinario:
   class Nodo {
     public:
      // Constructor:
      Nodo(const int dat, Nodo *izq=NULL, Nodo *der=NULL) :
        dato(dat), izquierdo(izq), derecho(der) {}
      // Miembros:
      int dato;
      Nodo *izquierdo;
      Nodo *derecho;
   };

   // Punteros de la lista, para cabeza y nodo actual:
   Nodo *raíz;
   Nodo *actual;
   int contador;
   int altura;

  public:
   // Constructor y destructor básicos:
   ArbolABB() : raíz(NULL), actual(NULL) {}
   ~ArbolABB() { Podar(raíz); }
   // Insertar en árbol ordenado:
   void Insertar(const int dat);
   // Borrar un elemento del árbol:
   void Borrar(const int dat);
   // Función de búsqueda:
   bool Buscar(const int dat);
   // Comprobar si el árbol está vacío:
   bool Vacio(Nodo *r) { return r==NULL; }
   // Comprobar si es un nodo hoja:
   bool EsHoja(Nodo *r) { return !r->derecho && !r->izquierdo; }
   // Contar número de nodos:
   const int NumeroNodos();
   const int AlturaArbol();
   // Calcular altura de un int:
   int Altura(const int dat);
   // Devolver referencia al int del nodo actual:
   int &ValorActual() { return actual->dato; }
```

```
   // Moverse al nodo raíz:
   void Raiz() { actual = raíz; }
   // Aplicar una función a cada elemento del árbol:
   void InOrden(void (*func)(int&) , Nodo *nodo=NULL, bool r=true);
   void PreOrden(void (*func)(int&) , Nodo *nodo=NULL, bool r=true);
   void PostOrden(void (*func)(int&) , Nodo *nodo=NULL, bool r=true);
  private:
   // Funciones auxiliares
   void Podar(Nodo* &);
   void auxContador(Nodo*);
   void auxAltura(Nodo*, int);
};
```

Definición de las funciones miembro:

Las definiciones de las funciones miembro de la clase no difieren demasiado de las que creamos en C. Tan solo se han sustituido algunos punteros por referencias, y se usa el tipo bool cuando es aconsejable.

Por ejemplo, en las funciones de recorrido de árboles, la función invocada acepta ahora una referencia a un entero, en lugar de un puntero a un entero.

Ejemplo de ABB en C++ empleando plantillas:

Sólo nos queda generalizar la clase del ejemplo anterior implementándola en forma de plantilla.

El proceso es relativamente sencillo, sólo tenemos que cambiar la declaración de dato, y todas sus referencias. Además de cambiar la sintaxis de las definiciones de las funciones, claro.

Declaración de la plantilla ArbolABB:

Se trata de una simple generalización de la clase del punto anterior:

```
template<class DATO>
class ABB {
  private:
   //// Clase local de Lista para Nodo de ArbolBinario:
   template<class DATON>
   class Nodo {
     public:
      // Constructor:
      Nodo(const DATON dat, Nodo<DATON> *izq=NULL, Nodo<DATON> *der=NULL) :
        dato(dat), izquierdo(izq), derecho(der) {}
      // Miembros:
      DATON dato;
      Nodo<DATON> *izquierdo;
      Nodo<DATON> *derecho;
   };

   // Punteros de la lista, para cabeza y nodo actual:
```

```
   Nodo<DATO> *raíz;
   Nodo<DATO> *actual;
   int contador;
   int altura;

  public:
   // Constructor y destructor básicos:
   ABB() : raíz(NULL), actual(NULL) {}
   ~ABB() { Podar(raíz); }
   // Insertar en árbol ordenado:
   void Insertar(const DATO dat);
   // Borrar un elemento del árbol:
   void Borrar(const DATO dat);
   // Función de búsqueda:
   bool Buscar(const DATO dat);
   // Comprobar si el árbol está vacío:
   bool Vacio(Nodo<DATO> *r) { return r==NULL; }
   // Comprobar si es un nodo hoja:
   bool EsHoja(Nodo<DATO> *r) { return !r->derecho && !r->izquierdo; }
   // Contar número de nodos:
   const int NumeroNodos();
   const int AlturaArbol();
   // Calcular altura de un dato:
   int Altura(const DATO dat);
   // Devolver referencia al dato del nodo actual:
   DATO &ValorActual() { return actual->dato; }
   // Moverse al nodo raíz:
   void Raiz() { actual = raíz; }
   // Aplicar una función a cada elemento del árbol:
   void InOrden(void (*func)(DATO&) , Nodo<DATO> *nodo=NULL, bool r=true);
   void PreOrden(void (*func)(DATO&) , Nodo<DATO> *nodo=NULL, bool r=true);
   void PostOrden(void (*func)(DATO&) , Nodo<DATO> *nodo=NULL, bool r=true);
  private:
   // Funciones auxiliares
   void Podar(Nodo<DATO>* &);
   void auxContador(Nodo<DATO>*);
   void auxAltura(Nodo<DATO>*, int);
};
```

Definición de las funciones miembro:

Las definiciones de las funciones miembro de la clase no difieren en nada de las que creamos en el ejemplo anterior. Tan solo se han sustituido los tipos del dato por el tipo de dato de la plantilla.

Árboles AVL:

Árboles Equilibrados:

Ya vimos que el comportamiento de los ABB no es siempre tan bueno como nos gustaría. Pues bien, para minimizar el problema de los ABB desequilibrados, sea cual sea el grado de desequilibrio que tengan, se puede recurrir a algoritmos de equilibrado de árboles globales. En cuanto a estos algoritmos, existen varios, por ejemplo, crear una lista mediante la lectura en inorden del árbol, y volver a reconstruirlo equilibrado. Conociendo el número de elementos no es demasiado complicado.

El problema de estos algoritmos es que requieren explorar y reconstruir todo el árbol cada vez que se inserta o se elimina un elemento, de modo que lo que ganamos al acortar las búsquedas, teniendo que hacer menos comparaciones, lo perdemos equilibrando el árbol.

Para resolver este inconveniente podemos recurrir a los árboles AVL.

Definición:

Un **árbol AVL** (llamado así por las iniciales de sus inventores: Adelson-Velskii y Landis) es un árbol binario de búsqueda en el que para cada nodo, las alturas de sus subárboles izquierdo y derecho no difieren en más de 1.

No se trata de árboles perfectamente equilibrados, pero sí son lo suficientemente equilibrados como para que su comportamiento sea lo bastante bueno como para usarlos donde los ABB no garantizan tiempos de búsqueda óptimos.

El algoritmo para mantener un árbol AVL equilibrado se basa en reequilibrados locales, de modo que no es necesario explorar todo el árbol después de cada inserción o borrado.

Operaciones en AVL:

Los AVL son también ABB, de modo que mantienen todas las operaciones que poseen éstos. Las nuevas operaciones son las de equilibrar el árbol, pero eso se hace como parte de las operaciones de insertado y borrado.

Factor de Equilibrio:

Cada nodo, además de la información que se pretende almacenar, debe tener los dos punteros a los árboles derecho e izquierdo, igual que los ABB, y además un miembro nuevo: el factor de equilibrio.

El factor de equilibrio es la diferencia entre las alturas del árbol derecho y el izquierdo:

FE = altura subárbol derecho - altura subárbol izquierdo;

Por definición, para un árbol AVL, este valor debe ser -1, 0 ó 1.

Los reequilibrados se realizan mediante rotaciones, en el siguiente punto veremos cada caso, ahora vamos a ver las cuatro posibles rotaciones que podemos aplicar.

Rotación simple a la derecha (SD):

Esta rotación se usará cuando el subárbol izquierdo de un nodo sea 2 unidades más alto que el derecho, es decir, cuando su FE sea de -2. Y además, la raíz del subárbol izquierdo tenga una FE de -1, es decir, que esté cargado a la izquierda.

Procederemos del siguiente modo:

Llamaremos P al nodo que muestra el desequilibrio, el que tiene una FE de -2. Y llamaremos Q al nodo raíz del subárbol izquierdo de P. Además, llamaremos A al subárbol izquierdo de Q, B al subárbol derecho de Q y C al subárbol derecho de P.

En el gráfico que puede observar que tanto B como C tienen la misma altura (n), y A es una unidad mayor (n+1). Esto hace que el FE de Q sea -1, la altura del subárbol que tiene Q como raíz es (n+2) y por lo tanto el FE de P es -2.

Pasamos el subárbol derecho del nodo Q como subárbol izquierdo de P. Esto mantiene el árbol como ABB, ya que todos los valores a la derecha de Q siguen estando a la izquierda de P.
El árbol P pasa a ser el subárbol derecho del nodo Q.
Ahora, el nodo Q pasa a tomar la posición del nodo P, es decir, hacemos que la entrada al árbol sea el nodo Q, en lugar del nodo P. Previamente, P puede que fuese un árbol completo o un subárbol de otro nodo de menor altura.

En el árbol resultante se puede ver que tanto P como Q quedan equilibrados en cuanto altura. En el caso de P porque sus dos subárboles tienen la misma altura (n), en el caso de Q, porque su subárbol izquierdo A tiene una altura (n+1) y su subárbol derecho también, ya que a P se añade la altura de cualquiera de sus subárboles.

Rotación simple a la izquierda (SI):

Se trata del caso simétrico del anterior. Esta rotación se usará cuando el subárbol derecho de un nodo sea 2 unidades más alto que el izquierdo, es decir, cuando su FE sea de 2. Y además, la raíz del subárbol derecho tenga una FE de 1, es decir, que esté cargado a la derecha.

Procederemos del siguiente modo:

Llamaremos P al nodo que muestra el desequilibrio, el que tiene una FE de 2. Y llamaremos Q al nodo raíz del subárbol derecho de P. Además, llamaremos A al subárbol izquierdo de P, B al subárbol izquierdo de Q y C al subárbol derecho de Q.

En el gráfico que puede observar que tanto A como B tienen la misma altura (n), y C es una unidad mayor (n+1). Esto hace que el FE de Q sea 1, la altura del subárbol que tiene Q como raíz es (n+2) y por lo tanto el FE de P es 2.

Pasamos el subárbol izquierdo del nodo Q como subárbol derecho de P. Esto mantiene el árbol como ABB, ya que todos los valores a la izquierda de Q siguen estando a la derecha de P.
El árbol P pasa a ser el subárbol izquierdo del nodo Q.
Ahora, el nodo Q pasa a tomar la posición del nodo P, es decir, hacemos que la entrada al árbol sea el nodo Q, en lugar del nodo P. Previamente, P puede que fuese un árbol completo o un subárbol de otro nodo de menor altura.

En el árbol resultante se puede ver que tanto P como Q quedan equilibrados en cuanto altura. En el caso de P porque sus dos subárboles tienen la misma altura (n), en el caso de Q, porque su subárbol izquierdo A tiene una altura (n+1) y su subárbol derecho también, ya que a P se añade la altura de cualquiera de sus subárboles.

Rotaciones dobles de nodos:

Rotación doble a la derecha (DD):

Esta rotación se usará cuando el subárbol izquierdo de un nodo sea 2 unidades más alto que el derecho, es decir, cuando su FE sea de -2. Y además, la raíz del subárbol izquierdo tenga una FE de 1, es decir, que esté cargado a la derecha.

Este es uno de los posibles árboles que pueden presentar esta estructura, pero hay otras dos posibilidades. El nodo R puede tener una FE de -1, 0 ó 1. En cada uno de esos casos los árboles izquierdo y derecho de R (B y C) pueden tener alturas de n y n-1, n y n, o n-1 y n, respectivamente.

El modo de realizar la rotación es independiente de la estructura del árbol R, cualquiera de las tres produce resultados equivalentes. Haremos el análisis para el caso en que FE sea -1.

En este caso tendremos que realizar dos rotaciones.

Llamaremos P al nodo que muestra el desequilibrio, el que tiene una FE de -2. Llamaremos Q al nodo raíz del subárbol izquierdo de P, y R al nodo raíz del subárbol derecho de Q.

Haremos una rotación simple de Q a la izquierda.
Después, haremos una rotación simple de P a la derecha.

Con más detalle, procederemos del siguiente modo:

Pasamos el subárbol izquierdo del nodo R como subárbol derecho de Q. Esto mantiene el árbol como ABB, ya que todos los valores a la izquierda de R siguen estando a la derecha de Q.

Ahora, el nodo R pasa a tomar la posición del nodo Q, es decir, hacemos que la raíz del subárbol izquierdo de P sea el nodo R en lugar de Q.
El árbol Q pasa a ser el subárbol izquierdo del nodo R.

Pasamos el subárbol derecho del nodo R como subárbol izquierdo de P. Esto mantiene el árbol como ABB, ya que todos los valores a la derecha de R siguen estando a la izquierda de P.
Ahora, el nodo R pasa a tomar la posición del nodo P, es decir, hacemos que la entrada al árbol sea el nodo R, en lugar del nodo P. Como en los casos anteriores, previamente, P puede que fuese un árbol completo o un subárbol de otro nodo de menor altura.
El árbol P pasa a ser el subárbol derecho del nodo R

Rotación doble a la izquierda (DI):

Esta rotación se usará cuando el subárbol derecho de un nodo sea 2 unidades más alto que el izquierdo, es decir, cuando su FE sea de 2. Y además, la raíz del subárbol derecho tenga una FE de -1, es decir, que esté cargado a la izquierda. Se trata del caso simétrico del anterior.

En este caso también tendremos que realizar dos rotaciones.

Llamaremos P al nodo que muestra el desequilibrio, el que tiene una FE de 2. Llamaremos Q al nodo raíz del subárbol derecho de P, y R al nodo raíz del subárbol izquierdo de Q.

Haremos una rotación simple de Q a la derecha.
Después, haremos una rotación simple de P a la izquierda.

Con más detalle, procederemos del siguiente modo:

Pasamos el subárbol derecho del nodo R como subárbol izquierdo de Q. Esto mantiene el árbol como ABB, ya que todos los valores a la derecha de R siguen estando a la izquierda de Q.
Ahora, el nodo R pasa a tomar la posición del nodo Q, es decir, hacemos que la raíz del subárbol derecho de P sea el nodo R en lugar de Q.
El árbol Q pasa a ser el subárbol derecho del nodo R.

Pasamos el subárbol izquierdo del nodo R como subárbol derecho de P. Esto mantiene el árbol como ABB, ya que todos los valores a la izquierda de R siguen estando a la derecha de P.
Ahora, el nodo R pasa a tomar la posición del nodo P, es decir, hacemos que la entrada al árbol sea el nodo R, en lugar del nodo P. Como en los casos anteriores, previamente, P puede que fuese un árbol completo o un subárbol de otro nodo de menor altura.
El árbol P pasa a ser el subárbol izquierdo del nodo R.

Reequilibrados en árboles en árboles AVL:

Cada vez que insertemos o eliminemos un nodo en un árbol AVL pueden suceder dos cosas: que el árbol se mantenga como AVL o que pierda esta propiedad. En el segundo caso siempre estaremos en uno de los explicados anteriormente, y recuperaremos el estado AVL aplicando la rotación adecuada.

Ya comentamos que necesitamos añadir un nuevo miembro a cada nodo del árbol para averiguar si el árbol sigue siendo AVL, el Factor de Equilibrio. Cada vez que insertemos o eliminemos un nodo deberemos recorrer el camino desde ese nodo hacia el nodo raíz actualizando los valores de FE de cada nodo. Cuando uno de esos valores sea 2 ó -2 aplicaremos la rotación correspondiente.

Debido a que debemos ser capaces de recorrer el árbol en dirección a la raíz, añadiremos un nuevo puntero a cada nodo que apunte al nodo padre. Esto complicará algo las operaciones de inserción, borrado y rotación, pero facilita y agiliza mucho el cálculo del FE, y veremos que las complicaciones se compensan en gran parte por las facilidades obtenidas al disponer de este puntero.

Nota: En rigor, no es necesario ese puntero, podemos almacenar el camino que recorremos para localizar un nodo concreto usando una pila, y después podemos usar la pila para recuperar el camino en orden inverso. Pero esto nos obliga a introducir otra estructura dinámica, y según mi opinión, complica en exceso el algoritmo.

Cuando estemos actualizando los valores de FE no necesitamos calcular las alturas de las dos ramas de cada nodo, sabiendo en valor anterior de FE, y sabiendo en qué rama hemos añadido o eliminado el nodo, es fácil calcular el nuevo valor de FE. Si el nodo ha sido añadido en la rama derecha o eliminado en la izquierda, y ha habido un cambio de altura en la rama, se incrementa el valor de FE; si el nodo ha sido añadido en la rama izquierda o eliminado en la derecha, y ha habido un cambio de altura en la rama, se decrementa el valor de FE.

Los cambios de altura en una rama se producen sólo cuando el FE del nodo raíz de esa rama ha cambiado de 0 a 1 ó de 0 a -1. En caso contrario, cuando el FE cambia de 1 a 0 ó de -1 a 0, no se produce cambio de altura.

Si no hay cambio de altura, los valores de FE del resto de los nodos hasta el raíz no pueden cambiar, recordemos que el factor de equilibrio se define como la diferencia de altura entre las ramas derecha e izquierda de un nodo, la altura de la rama que no pertenece al camino no puede cambiar, puesto que sigue teniendo los mismos nodos que antes, de modo que si la altura de la rama que pertenece al camino no cambia, tampoco puede cambiar el valor de FE.

Por ejemplo, supongamos que en siguiente árbol AVL insertamos el nodo de valor 8:

Para empezar, cualquier nodo nuevo será un nodo hoja, de modo que su FE será siempre 0.

Ahora actualizamos el valor de FE del nodo padre del que acabamos de insertar (P). El valor previo es 0, y hemos añadido un nodo en su rama izquierda, por lo tanto, el nuevo valor es -1. Esto implica un cambio de altura, por lo tanto, continuamos camino hacia la raíz.

A continuación tomamos el nodo padre de P (Q), cuyo valor previo de FE era 1, y al que también hemos añadido un nodo en su rama izquierda, por lo tanto decrementamos ese valor, y el nuevo será 0. En este caso no ha incremento de altura, la altura del árbol cuya raíz es Q sigue siendo la misma, por lo tanto, ninguno de los valores de FE de los nodos hasta el raíz puede haber cambiado. Es decir, no necesitamos seguir recorriendo el camino.

Si verificamos el valor de FE del nodo R vemos que efectivamente se mantiene, puesto que tanto la altura del subárbol derecho como del izquierdo, siguen siendo las mismas.

Pero algunas veces, el valor de FE del nodo es -2 ó 2, son los casos en los que perdemos la propiedad AVL del árbol, y por lo tanto tendremos que recuperarla.

Reequilibrados en árboles AVL por inserción de un nodo

En ese caso, cuando el valor de FE de un nodo tome el valor -2 ó 2, no seguiremos el camino, sino que, con el valor de FE de el nodo actual y el del nodo derecho si FE es 2 o el del nodo izquierdo si es -2, determinaremos qué tipo de rotación debemos hacer.

FE nodo actual	FE del nodo derecho	FE del nodo izquierdo	Rotación
-2	No importa	-1	RSD
-2	No importa	1	RDD
2	-1	No importa	RDI
2	1	No importa	RSI

El resto de los casos no nos interesan. Esto es porque en nodos desequilibrados hacia la derecha, con valores de FE positivos, siempre buscaremos el equilibrio mediante rotaciones a la izquierda, y viceversa, con nodos desequilibrados hacia la izquierda, con valores de FE negativos, buscaremos el equilibrio mediante rotaciones a la derecha.

Supongamos que el valor de FE del nodo ha pasado de -1 a -2, debido a que se ha añadido un nodo. Esto implica que el nodo añadido lo ha sido en la rama izquierda, si lo hubiéramos añadido en la derecha el valor de FE nunca podría decrecer.

Reequilibrados en árboles AVL por borrado de un nodo:

Cuando el desequilibrio se debe a la eliminación de un nodo la cosa puede ser algo diferente, pero veremos que siempre se puede llegar a uno de los casos anteriores.

Supongamos el siguiente ejemplo, en el árbol AVL eliminaremos el nodo de valor 3:

El valor de FE del nodo P pasa de 1 a 2, sabemos que cuando el valor de FE de un nodo es 2 siempre tenemos que aplicar una rotación a izquierdas. Para saber cual de las dos rotaciones debemos aplicar miramos el valor de FE del nodo derecho. Pero en este caso, el valor de FE de ese nodo es 0. Esto no quiere decir que no podamos aplicar ninguna de las rotaciones, por el contrario, podremos aplicar cualquiera de ellas. Aunque por economía, lo razonable es aplicar la rotación simple.

Del mismo modo, el valor de FE del nodo derecho podría haber sido 1 ó -1, en ese caso sí está determinado el tipo de rotación a realizar.

El razonamiento es similar cuando se eliminan nodos y el resultado es que se obtiene un nodo con FE de -2, en este caso se realizará una rotación a derechas, y la rotación dependerá del valor de FE del nodo izquierdo al que muestra el desequilibrio. Si es 0 ó -1 haremos una rotación simple, si es 1, haremos una rotación doble.

Tendremos entonces una tabla más general para decidir la rotación a aplicar:

FE nodo actual	FE del nodo derecho	FE del nodo izquierdo	Rotación
-2	No importa	-1	RSD

-2	No importa	0	RSD
-2	No importa	1	RDD
2	-1	No importa	RDI
2	0	No importa	RSI
2	1	No importa	RSI

Los árboles AVL siempre quedan equilibrados después de una rotación:

Esto puede comprobarse analizando los métodos de rotación que hemos estudiado, después de efectuada la rotación, la altura del árbol cuya raíz es el nodo rotado se mantiene, por lo tanto, no necesitamos continuar el camino hacia la raíz: sabemos que el árbol es AVL.

Algoritmos:

De inserción de nodo:

En general, la inserción de nodos en un árbol AVL es igual que en un árbol ABB, la diferencia es que en un árbol AVL, después de insertar el nodo debemos recorrer el árbol en sentido hacia la raíz, recalculando los valores de FE, hasta que se cumpla una de estas condiciones: que lleguemos a la raíz, que se encuentre un nodo con valor de FE de 2, ó -2, o que se llegue a un nodo cuyo FE no cambie o decrezca en valor absoluto, es decir, que cambie de 1 a 0 ó de -1 a 0.

Podemos considerar que el algoritmo de inserción de nodos en árboles AVL es una ampliación del que vimos para árboles ABB.

De borrado de nodo:

Lo mismo pasa cuando se eliminan nodos, el algoritmo es el mismo que en árboles ABB, pero después de eliminar el nodo debemos recorrer el camino hacia la raíz recalculando los valores de FE, y equilibrando el árbol si es necesario.

De recalcular FE:

Ya comentamos más atrás que para seguir el camino desde el nodo insertado o borrado hasta el nodo raíz tenemos dos alternativas:

Guardar en una pila los punteros a los nodos por los que hemos pasado para llegar al nodo insertado o borrado, es decir, almacenar el camino.

Añadir un nuevo puntero a cada nodo que apunte al padre del nodo actual. Esto nos permite recorrer el árbol en el sentido contrario al normal, es decir, en dirección a la raíz.

Para calcular los nuevos valores de FE de los nodos del camino hay que tener en cuenta los siguientes hechos:

El valor de FE de un nodo insertado es cero, ya que siempre insertaremos nodos hoja.
Si el nuevo valor de FE para cualquiera de los siguientes nodos del camino es cero, habremos terminado de actualizar los valores de FE, ya que la rama mantiene su altura, la inserción o borrado del nodo no puede influir en los valores de FE de los siguientes nodos del camino.
Cuando se elimine un nodo pueden pasar dos cosas. Siempre eliminamos un nodo hoja, ya que cuando no lo es, lo intercambiamos con un nodo hoja antes de eliminarlo. Pero algunas veces, el nodo padre del nodo eliminado se convertirá a su vez en nodo hoja, y en ese caso no siempre hay que dar por terminada la actualización del FE del camino. Por lo tanto, cuando eliminemos un nodo, actualizaremos el valor de FE del nodo padre y continuaremos el camino, independientemente del valor de FE calculado.
A la hora de actualizar el valor de FE de un nodo, tenemos que distinguir cuando el equilibrado sea consecuencia de una inserción o lo sea de una eliminación. Incrementaremos el valor de FE del nodo si la inserción fue en la rama derecha o si la eliminación fue en la rama izquierda, decrementaremos si la inserción fue en la izquierda o la eliminación en la derecha.
Si en valor de FE es -2, haremos una rotación doble a la derecha su el valor de FE del nodo izquierdo es 1, y simple si es 1 ó 0.
Si en valor de FE es 2, haremos una rotación doble a la izquierda su el valor de FE del nodo izquierdo es -1, y simple si es -1 ó 0.
En cualquiera de los dos casos, podremos dar por terminado el recorrido del camino, ya que la altura del árbol cuya raíz es un nodo rotado no cambia.
En cualquier otro caso, seguiremos actualizando hasta llegar al nodo raíz.

De rotación simple:

A la hora de implementar los algoritmos que hemos visto para rotaciones simples tenemos dos opciones: seguir literalmente los pasos de los gráficos, o tomar un atajo, y hacerlo mediante asignaciones. Nosotros lo haremos del segundo modo, ya que resulta mucho más rápido y sencillo.

Primero haremos las reasignaciones de punteros, de modo que el árbol resultante responda a la estructura después de la rotación. Después actualizaremos los punteros al nodo padre para los nodos que han cambiado de posición. Por último actualizaremos los valores de FE de esos mismos nodos.

Para la primera fase usaremos punteros auxiliares a nodo, que en el caso de rotación a la derecha necesitamos un puntero P al nodo con FE igual a -2. Ese será el parámetro de entrada, otro puntero al nodo izquierdo de P: Q. Y tres punteros más a los árboles A, B y C.

En realidad, si nos fijamos en los gráficos, los punteros a A y C no son necesarios, ya que ambos conservan sus posiciones, A sigue siendo el subárbol izquierdo de Q y C el subárbol derecho de P.

Usaremos otro puntero más: Padre, que apunte al padre de P. Disponiendo de los punteros Padre, P, Q y B, realizar la rotación es muy sencillo:

```
if(Padre)
  if(Padre->derecho == P) Padre->derecho = Q;
```

```
     else Padre->izquierdo = Q;
   else raíz = Q;

   // Reconstruir árbol:
   P->izquierdo = B;
   Q->derecho = P;
```

Hay que tener en cuenta que P puede ser la raíz de un subárbol derecho o izquierdo de otro nodo, o incluso la raíz del árbol completo. Por eso comprobamos si P tiene padre, y si lo tiene, cual de sus ramas apunta a P, cuando lo sabemos, hacemos que esa rama apunte a Q. Si Padre es NULL, entonces P era la raíz del árbol, así que hacemos que la nueva raíz sea Q.

Sólo nos queda trasladar el subárbol B a la rama izquierda de P, y Q a la rama derecha de P.

La segunda fase consiste en actualizar los punteros padre de los nodos que hemos cambiado de posición: P, B y Q.

```
   P->padre = Q;
   if(B) B->padre = P;
   Q->padre = Padre;
```

El padre de P es ahora Q, el de Q es Padre, y el de B, si existe es P.

La tercera fase consiste en ajustar los valores de FE de los nodos para los que puede haber cambiado.

Esto es muy sencillo, después de una rotación simple, los únicos valores de FE que cambian son los de P y Q, y ambos valen 0.

```
// Rotación simple a derechas
void RSD(Nodo* nodo)
{
   Nodo *Padre = nodo->padre;
   Nodo *P = nodo;
   Nodo *Q = P->izquierdo;
   Nodo *B = Q->derecho;

   if(Padre)
     if(Padre->derecho == P) Padre->derecho = Q;
     else Padre->izquierdo = Q;
   else raíz = Q;

   // Reconstruir árbol:
   P->izquierdo = B;
   Q->derecho = P;

   // Reasignar padres:
   P->padre = Q;
   if(B) B->padre = P;
   Q->padre = Padre;

   // Ajustar valores de FE:
   P->FE = 0;
   Q->FE = 0;
}
```

La rotación a izquierdas es simétrica.

De rotación doble:

Para implementar las rotaciones dobles trabajaremos de forma análoga.

Primero haremos las reasignaciones de punteros, de modo que el árbol resultante responda a la estructura después de la rotación. Después actualizaremos los punteros al nodo padre para los nodos que han cambiado de posición. Por último actualizaremos los valores de FE de esos mismos nodos.

Para la primera fase usaremos punteros auxiliares a nodo, que en el caso de rotación a la derecha necesitamos un puntero P al nodo con FE igual a -2. Ese será el parámetro de entrada, otro puntero al nodo izquierdo de P: Q. Un tercero al nodo derecho de Q: R. Y cuatro punteros más a los árboles A, B, C y D.

En realidad, si nos fijamos en los gráficos, los punteros a A y D no son necesarios, ya que ambos conservan sus posiciones, A sigue siendo el subárbol izquierdo de Q y D el subárbol derecho de P.

También en este caso usaremos otro puntero más: Padre, que apunte al padre de P. Disponiendo de los punteros Padre, P, Q, R, B y C, realizar la rotación es muy sencillo:

```
if(Padre)
  if(Padre->derecho == nodo) Padre->derecho = R;
  else Padre->izquierdo = R;
else raíz = R;

// Reconstruir árbol:
Q->derecho = B;
P->izquierdo = C;
R->izquierdo = Q;
R->derecho = P;
```

Ahora también hay que tener en cuenta que P puede ser la raíz de un subárbol derecho o izquierdo de otro nodo, o incluso la raíz del árbol completo. Por eso comprobamos si P tiene padre, y si lo tiene, cual de sus ramas apunta a P, cuando lo sabemos, hacemos que esa rama apunte a R. Si Padre es NULL, entonces P era la raíz del árbol, así que hacemos que la nueva raíz sea R.

Sólo nos queda trasladar el subárbol B a la rama derecha de Q, C a la rama izquierda de P, Q a la rama izquierda de R y P a la rama derecha de R.

La segunda fase consiste en actualizar los punteros padre de los nodos que hemos cambiado de posición: P, Q, R, B y C.

```
R->>padre = Padre;
P->padre = Q->padre = R;
if(B) B->padre = Q;
if(C) C->padre = P;
```

El padre de R es ahora Padre, el de P y Q es R, y el de B, si existe es Q, y el de C, si existe, es P.

La tercera fase consiste en ajustar los valores de FE de los nodos para los que puede haber cambiado.

En las rotaciones dobles esto se complica un poco ya que puede suceder que el valor de FE de R antes de la rotación sea -1, 0 o 1. En cada caso, los valores de FE de P y Q después de la rotación serán diferentes.

```
   // Ajustar valores de FE:
   switch(R->FE) {
      case -1: Q->FE = 0; P->FE = 1; break;
      case 0:  Q->FE = 0; P->FE = 0; break;
      case 1:  Q->FE = -1; P->FE = 0; break;
   }
   R->FE = 0;
```

Si la altura de B es n-1 y la de C es n, el valor de FE de R es 1. Después de la rotación, la rama B pasa a ser el subárbol derecho de Q, por lo tanto, la FE de Q, dado que la altura de su rama izquierda es n, será 0. La rama C pasa a ser el subárbol izquierdo de P, y dado que la altura de la rama derecha es n, la FE de P será -1.

Si la altura de B es n y la de C es n-1, el valor de FE de R es -1. Después de la rotación, la rama B pasa a ser el subárbol derecho de Q, por lo tanto, la FE de Q, dado que la altura de su rama izquierda es n, será 0. La rama C pasa a ser el subárbol izquierdo de P, y dado que la altura de la rama derecha es n, la FE de P será 0.

Por último, si la altura de B y C es n, el valor de FE de R es 0. Después de la rotación, la rama B pasa a ser el subárbol derecho de Q, por lo tanto, la FE de Q, dado que la altura de su rama izquierda es n, será 0. La rama C pasa a ser el subárbol izquierdo de P, y dado que la altura de la rama derecha es n, la FE de P será 0.

```
// Rotación doble a derechas
void RDD(Nodo* nodo)
{
   Nodo *Padre = nodo->padre;
   Nodo *P = nodo;
   Nodo *Q = P->izquierdo;
   Nodo *R = Q->derecho;
   Nodo *B = R->izquierdo;
   Nodo *C = R->derecho;

   if(Padre)
     if(Padre->derecho == nodo) Padre->derecho = R;
     else Padre->izquierdo = R;
   else raíz = R;

   // Reconstruir árbol:
   Q->derecho = B;
   P->izquierdo = C;
   R->izquierdo = Q;
   R->derecho = P;

   // Reasignar padres:
   R->padre = Padre;
   P->padre = Q->padre = R;
   if(B) B->padre = Q;
   if(C) C->padre = P;
```

```
   // Ajustar valores de FE:
   switch(R->FE) {
      case -1: Q->FE = 0; P->FE = 1; break;
      case 0:  Q->FE = 0; P->FE = 0; break;
      case 1:  Q->FE = -1; P->FE = 0; break;
   }
   R->FE = 0;
}
```

Ejemplo de árbol AVL en C:

No ha demasiado que añadir, construiremos los ejemplos de árboles AVL basándonos en los ejemplos que hicimos para árboles binarios de búsqueda.

Sólo tenemos que añadir cinco nuevas funciones: una para equilibrar el árbol, y cuatro para las cuatro posibles rotaciones.

Además, modificaremos ligeramente las funciones de inserción y borrado para que se equilibre el árbol automáticamente después de cada inserción o borrado.

En la estructura de nodo para árbol AVL añadiremos un puntero al nodo padre y un valor entero para almacenar el factor de equilibrio.

Cuando se inserten nuevos nodos hay que ajustar los valores de los nuevos miembros de nodo: FE y padre. Seguidamente, llamaremos a la función "Equilibrar", salvo que el nodo insertado sea el raíz, ya que en ese caso no es necesario, evidentemente.

El procedimiento de equilibrar consiste en la implementación del algoritmo que vimos en el punto anterior:

```
/* Equilibrar árbol AVL partiendo de un nodo*/
void Equilibrar(Arbol *a, pNodo nodo, int rama, int nuevo)
{
   int salir = FALSE;

   /* Recorrer camino inverso actualizando valores de FE: */
   while(nodo && !salir) {
      if(nuevo)
         if(rama == IZQUIERDO) nodo->FE--; /* Depende de si añadimos ... */
         else                  nodo->FE++;
      else
         if(rama == IZQUIERDO) nodo->FE++; /* ... o borramos */
         else                  nodo->FE--;
      if(nodo->FE == 0) salir = TRUE; /* La altura de las rama que
                                         empieza en nodo no ha variado,
                                         salir de Equilibrar */
      else if(nodo->FE == -2) { /* Rotar a derechas y salir: */
         if(nodo->izquierdo->FE == 1) RDD(a, nodo); /* Rotación doble  */
         else RSD(a, nodo);                         /* Rotación simple */
         salir = TRUE;
      }
      else if(nodo->FE == 2) {  /* Rotar a izquierdas y salir: */
         if(nodo->derecho->FE == -1) RDI(a, nodo); /* Rotación doble  */
         else RSI(a, nodo);                        /* Rotación simple */
         salir = TRUE;
```

```
      }
      if(nodo->padre)
         if(nodo->padre->derecho == nodo) rama = DERECHO; else rama = IZQUIERDO;
      nodo = nodo->padre; /* Calcular FE, siguiente nodo del camino. */
   }
}
```

Las funciones para rotar son también sencillas, por ejemplo, la rotación simple a la derecha:

```
/* Rotación simple a derechas */
void RSD(Arbol *a, pNodo nodo)
{
   pNodo Padre = nodo->padre;
   pNodo P = nodo;
   pNodo Q = P->izquierdo;
   pNodo B = Q->derecho;

   if(Padre)
     if(Padre->derecho == P) Padre->derecho = Q;
     else Padre->izquierdo = Q;
   else *a = Q;

   /* Reconstruir árbol: */
   P->izquierdo = B;
   Q->derecho = P;

   /* Reasignar padres: */
   P->padre = Q;
   if(B) B->padre = P;
   Q->padre = Padre;

   /* Ajustar valores de FE: */
   P->FE = 0;
   Q->FE = 0;
}
```

Y la rotación doble a la derecha:

```
/* Rotación doble a derechas */
void RDD(Arbol *raíz, Arbol nodo)
{
   pNodo Padre = nodo->padre;
   pNodo P = nodo;
   pNodo Q = P->izquierdo;
   pNodo R = Q->derecho;
   pNodo B = R->izquierdo;
   pNodo C = R->derecho;

   if(Padre)
     if(Padre->derecho == nodo) Padre->derecho = R;
     else Padre->izquierdo = R;
   else *raíz = R;

   /* Reconstruir árbol: */
   Q->derecho = B;
```

```
   P->izquierdo = C;
   R->izquierdo = Q;
   R->derecho = P;

   /* Reasignar padres: */
   R->padre = Padre;
   P->padre = Q->padre = R;
   if(B) B->padre = Q;
   if(C) C->padre = P;

   /* Ajustar valores de FE: */
   switch(R->FE) {
      case -1: Q->FE = 0; P->FE = 1; break;
      case 0:  Q->FE = 0; P->FE = 0; break;
      case 1:  Q->FE = -1; P->FE = 0; break;
   }
   R->FE = 0;
}
```

Bibliografía:

Es muy aconsejable el uso del texto:

Programación en C. Metodología, estructura de datos y objetos.

Luis Joyanes Aguilar e Ignacio Zahonero Martínez.
Editorial: McGraw-Hill

Tutorial bajado de:

http://programandoenc.webcindario.com